AIMS Systems Verification
Quantitative Verification Part 1

Prof. Marta Kwiatkowska

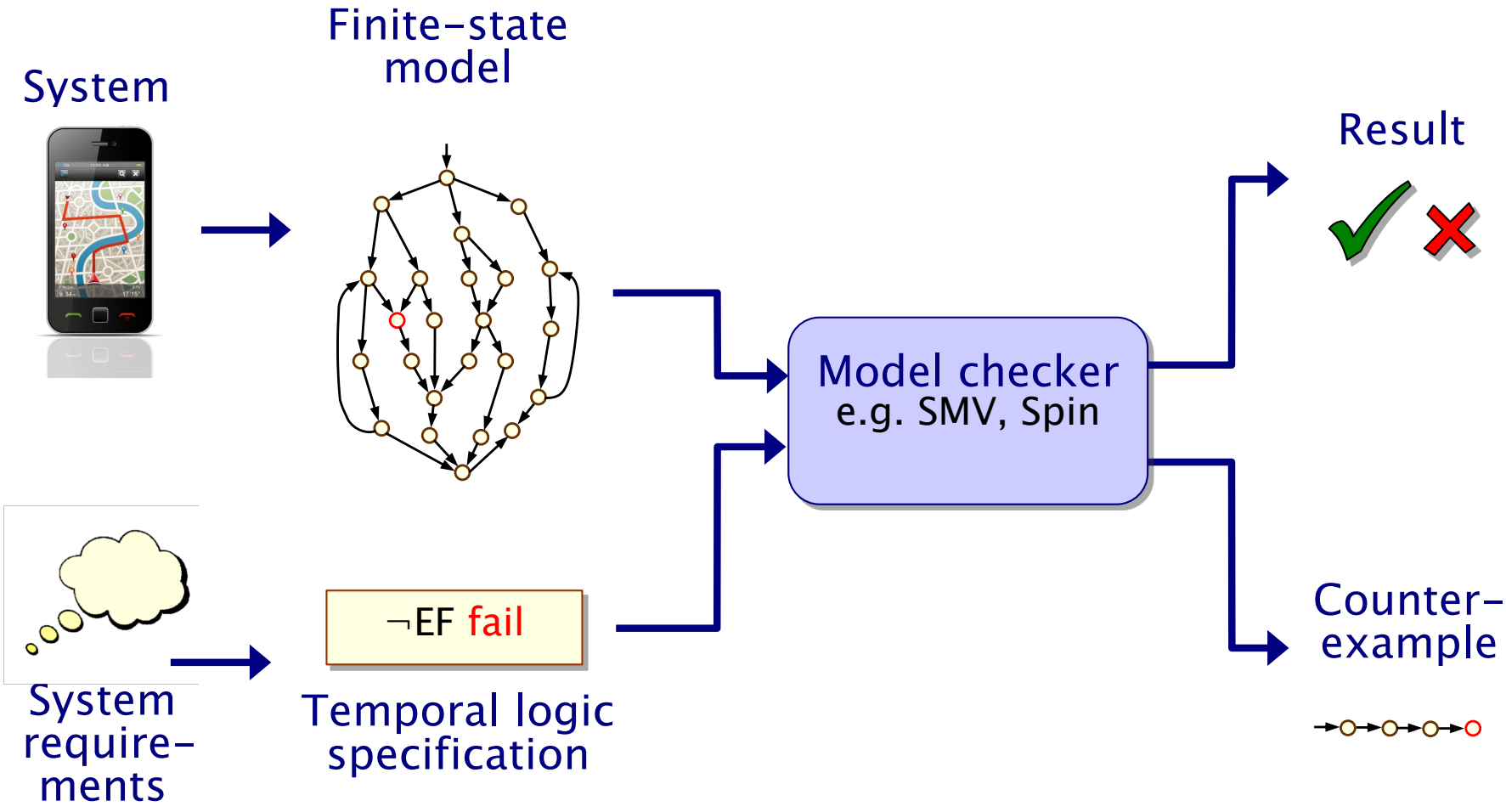


Department of Computer Science
University of Oxford

What is quantitative verification?

- Quantitative verification…
  - is a formal verification technique for modelling and analysing quantitative aspect of probabilistic systems
  - also called probabilistic model checking
- Formal verification (aka model checking)…
  - is the application of rigorous, mathematics-based techniques to establish the correctness of computerised systems

Verification via model checking

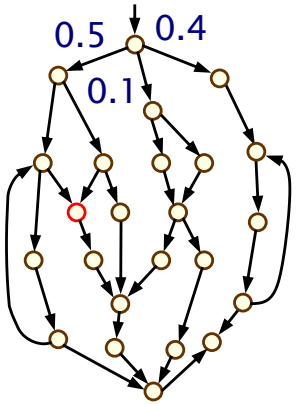


Quantitative verification

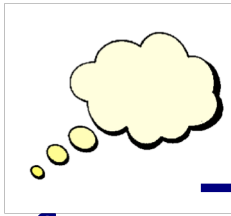
System



Probabilistic model
e.g. Markov chain



System require-ments

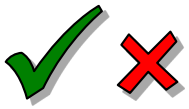


$P_{<0.1}$ [F fail]

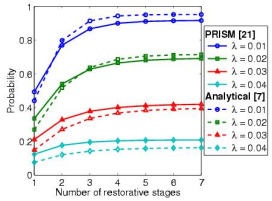
Probabilistic temporal logic specification
e.g. PCTL, CSL, LTL

Probabilistic model checker
e.g. PRISM

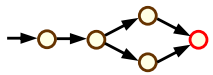
Result



Quantitative results



Counter-example



Why probability?

- Some systems are inherently probabilistic…
- Randomisation, e.g. in wireless coordination protocols
  - as a symmetry breaker

    bool short_delay = Bernoulli(0.5) // short or long delay
- Modelling uncertainty
  - to quantify rate of failures

    bool fail = Bernoulli(0.001) // success wp 0.999 or failure
- Modelling performance
  - queuing systems are naturally modelled in a stochastic fashion

    float arrival_rate = exp(2.5) // exponentially distributed

Verifying probabilistic systems

- We are not just interested in correctness

- We want to be able to quantify:
  - security, privacy, trust, anonymity, fairness
  - safety, reliability, performance, dependability
  - resource usage, e.g. battery life
  - and much more…

- Quantitative, as well as qualitative requirements:
  - how reliable is my car's Bluetooth network?
  - how efficient is my phone's power management policy?
  - is my bank's web-service secure?
  - what is the expected long-run percentage utilisation of resource X?

Probabilistic models

	Fully probabilistic	Nondeterministic
Discrete time	Discrete-time Markov chains (DTMCs)	Markov decision processes (MDPs)
		Simple stochastic games (SMGs)
Continuous time	Continuous-time Markov chains (CTMCs)	Probabilistic timed automata (PTAs)
		Interactive Markov chains (IMCs)

NB can also consider continuous space…

Course material

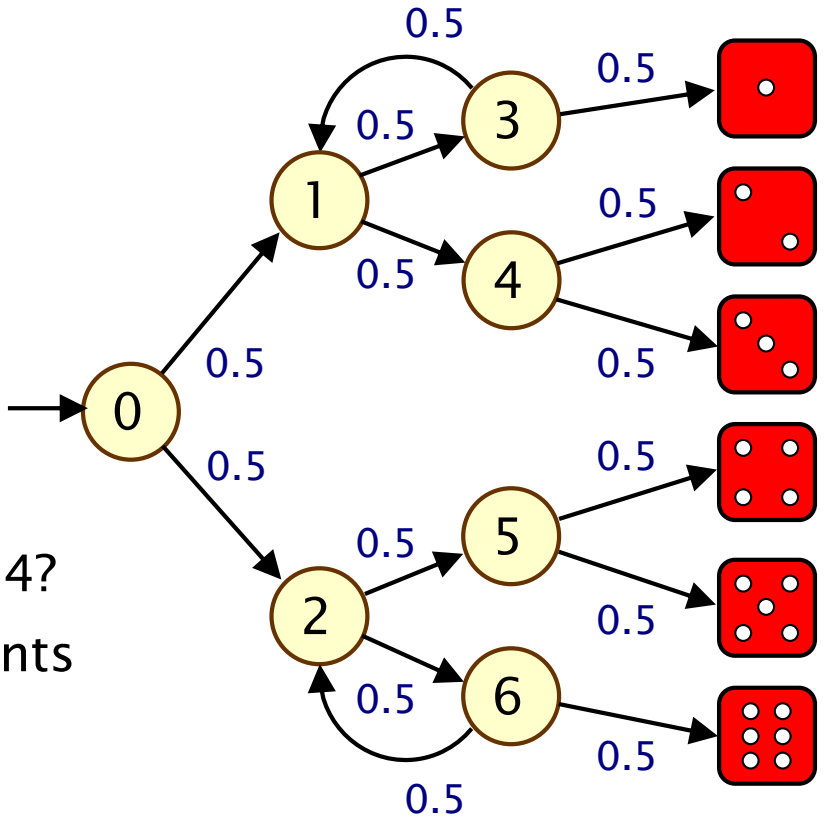
- Quantitative Verification lecture slides and lab session
  - http://www.prismmodelchecker.org/courses/aims1819/
- Reading
  - [DTMCs/CTMCs] Kwiatkowska, Norman and Parker. Stochastic Model Checking. LNCS vol 4486, p220-270, Springer 2007.
  - [MDPs/LTL] Forejt, Kwiatkowska, Norman and Parker. Automated Verification Techniques for Probabilistic Systems. LNCS vol 6659, p53-113, Springer 2011.
  - [DTMCs/MDPs/LTL] Principles of Model Checking by Baier and Katoen, MIT Press 2008
- See also
  - 20 lecture course taught at Oxford
  - http://www.prismmodelchecker.org/lectures/pmc/
- PRISM website www.prismmodelchecker.org

Overview (Part 1)

- Probability basics
- Discrete-time Markov chains (DTMCs)
  - definition, paths & probability spaces
- Temporal logic PCTL
- PCTL model checking
- Costs and rewards
- PRISM: overview
  - Modelling language
  - Properties
  - GUI, etc
  - Case study: Bluetooth device discovery
- Summary

Probability example

- Modelling a 6-sided die using a fair coin
  - algorithm due to Knuth/Yao:
  - start at 0, toss a coin
  - upper branch when H
  - lower branch when T
  - repeat until value chosen

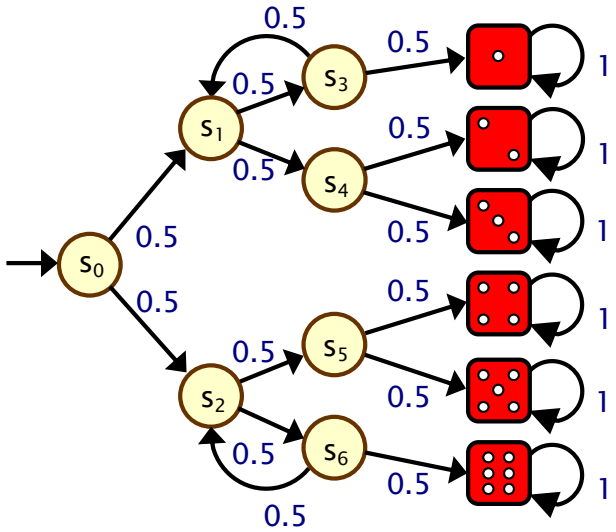


- Is this algorithm correct?
  - e.g. probability of obtaining a 4?
  - obtain as disjoint union of events
  - THH, TTTHH, TTTTTHH, …
  - Pr("eventually 4")

    $= (1/2)^3 + (1/2)^5 + (1/2)^7 + \ldots = 1/6$

Example…

- Other properties?
  - "what is the probability of termination?"
- e.g. efficiency?
  - "what is the probability of needing more than 4 coin tosses?"
  - "on average, how many coin tosses are needed?"

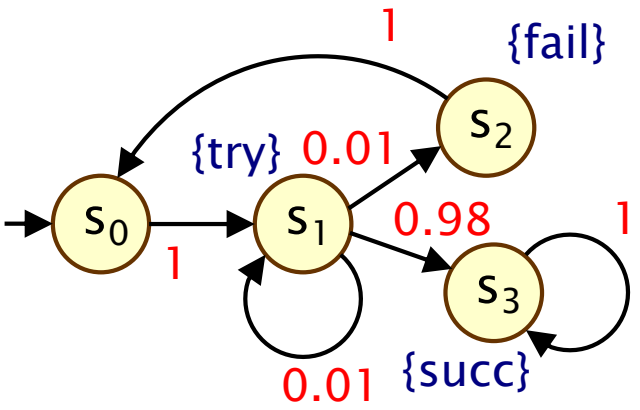


- Probabilistic model checking provides a framework for these kinds of properties…
  - modelling languages
  - property specification languages
  - model checking algorithms, techniques and tools

Discrete-time Markov chains

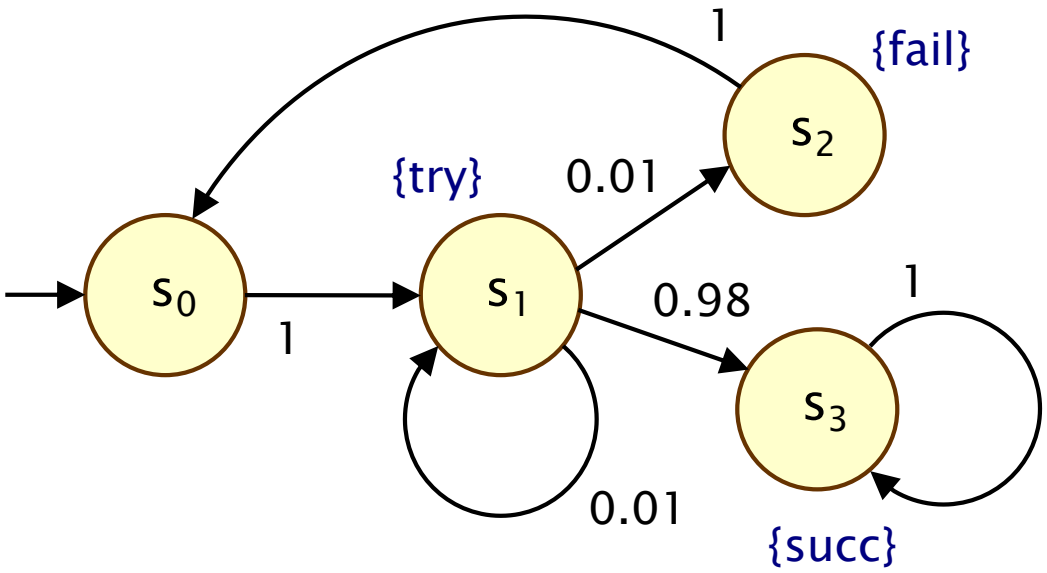
- State-transition systems augmented with probabilities

- States
  - set of states representing possible configurations of the system being modelled
- Transitions
  - transitions between states model evolution of system's state; occur in discrete time-steps
- Probabilities
  - probabilities of making transitions between states are given by discrete probability distributions



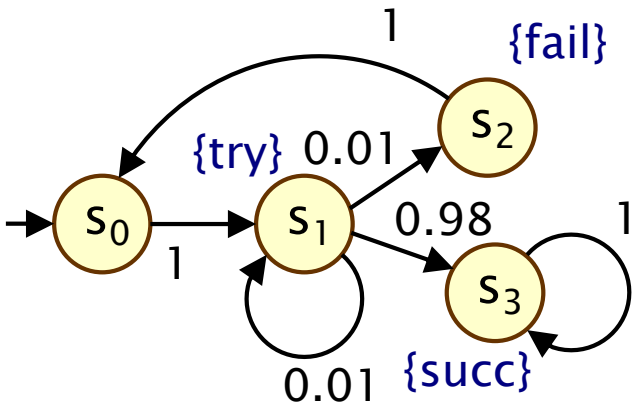
Simple DTMC example

- Modelling a very simple communication protocol
  - after one step, process starts trying to send a message
  - with probability 0.01, channel unready so wait a step
  - with probability 0.98, send message successfully and stop
  - with probability 0.01, message sending fails, restart



Discrete-time Markov chains

- Formally, a DTMC D is a tuple $(S, s_{init}, \mathbf{P}, L)$ where:
  - S is a set of states ("state space")
  - $s_{init} \in S$ is the initial state
  - $\mathbf{P} : S \times S \rightarrow [0,1]$ is the transition probability matrix where $\Sigma_{s' \in S} \mathbf{P}(s,s') = 1$ for all $s \in S$
  - $L : S \rightarrow 2^{AP}$ is function labelling states with atomic propositions (taken from a set AP)



Simple DTMC example

$D = (S, s_{init}, \mathbf{P}, L)$

$S = \{s_0, s_1, s_2, s_3\}$

$s_{init} = s_0$

$AP = \{try, fail, succ\}$

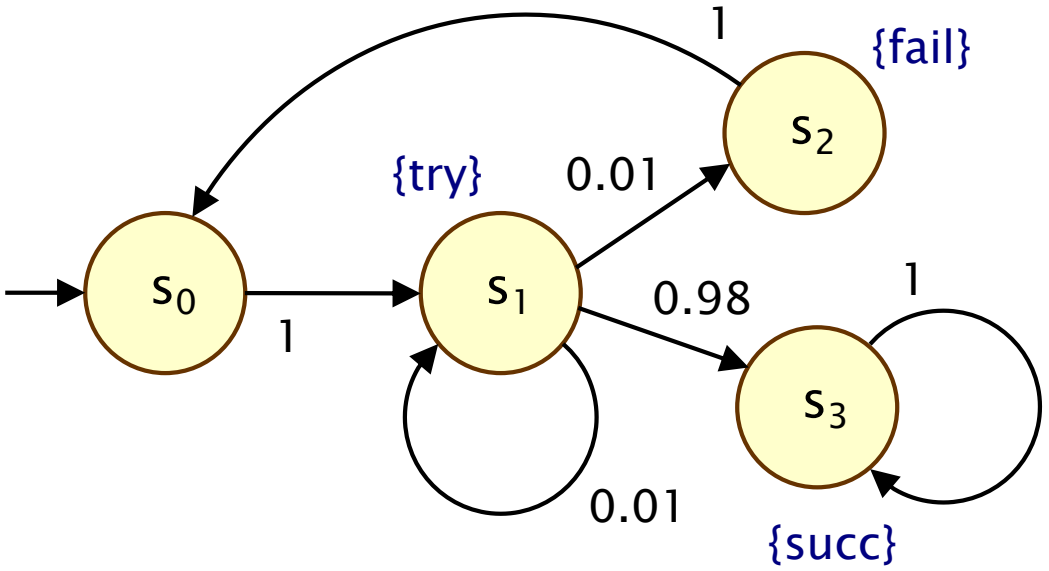
$L(s_0) = \emptyset$,

$L(s_1) = \{try\}$,

$L(s_2) = \{fail\}$,

$L(s_3) = \{succ\}$

$$\mathbf{P} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0.01 & 0.01 & 0.98 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Some more terminology

- **P** is a stochastic matrix, meaning it satisfies:
  - $\mathbf{P}(s,s') \in [0,1]$ for all $s,s' \in S$ and $\Sigma_{s' \in S} \mathbf{P}(s,s') = 1$ for all $s \in S$
- A sub-stochastic matrix satisfies:
  - $\mathbf{P}(s,s') \in [0,1]$ for all $s,s' \in S$ and $\Sigma_{s' \in S} \mathbf{P}(s,s') \leq 1$ for all $s \in S$
- An absorbing state is a state s for which:
  - $\mathbf{P}(s,s) = 1$ and $\mathbf{P}(s,s') = 0$ for all $s \neq s'$
  - the transition from s to itself is sometimes called a self-loop
- Note: Since we assume **P** is stochastic…
  - every state has at least one outgoing transition
  - i.e. no deadlocks (in model checking terminology)

DTMCs: An alternative definition

- Alternative definition… a DTMC is:
  - a family of random variables $\{ X(k) \mid k=0,1,2,\ldots \}$
  - where X(k) are observations at discrete time-steps
  - i.e. X(k) is the state of the system at time-step k
  - which satisfies…

- The Markov property ("memorylessness")
  - $\Pr(X(k)=s_k \mid X(k-1)=s_{k-1}, \ldots , X(0)=s_0)$
    $= \Pr(X(k)=s_k \mid X(k-1)=s_{k-1})$
  - for a given current state, future states are independent of past

- This allows us to adopt the "state-based" view presented so far (which is better suited to this context)

Other assumptions made here

- We consider time-homogenous DTMCs
  - transition probabilities are independent of time
  - for all k>0 and all states s,s':
  - **P**(s,s') = Pr(X(k)=s' | X(k-1)=s)
  - otherwise: time-inhomogenous

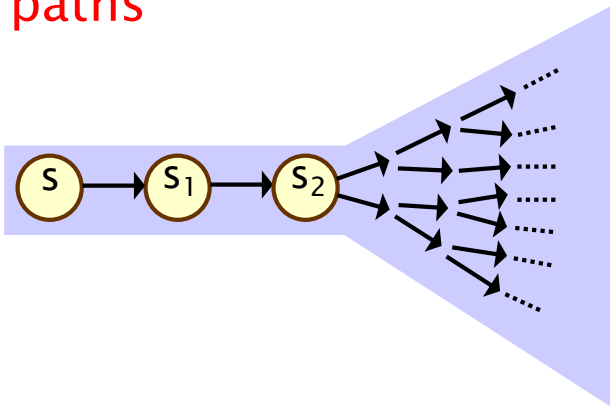
- We will (mostly) assume that the state space S is finite
  - in general, S can be any countable set

- Initial state $s_{init} \in S$ can be generalised…
  - to an initial probability distribution $s_{init} : S \rightarrow [0,1]$

- Focus on path-based properties
  - rather than steady-state

Paths and probabilities

- A (finite or infinite) path through a DTMC
  - is a sequence of states $s_0 s_1 s_2 s_3 \ldots$ such that $\mathbf{P}(s_i, s_{i+1}) > 0 \ \forall i$
  - represents an execution (i.e. one possible behaviour) of the system which the DTMC is modelling
- To reason (quantitatively) about this system
  - need to define a probability space over paths
- Intuitively:
  - sample space: Path(s) = set of all infinite paths from a state s
  - events: sets of infinite paths from s
  - basic events: cylinder sets (or "cones")
  - cylinder set C(ω), for a finite path ω = set of infinite paths with the common finite prefix ω
  - for example: $C(ss_1s_2)$



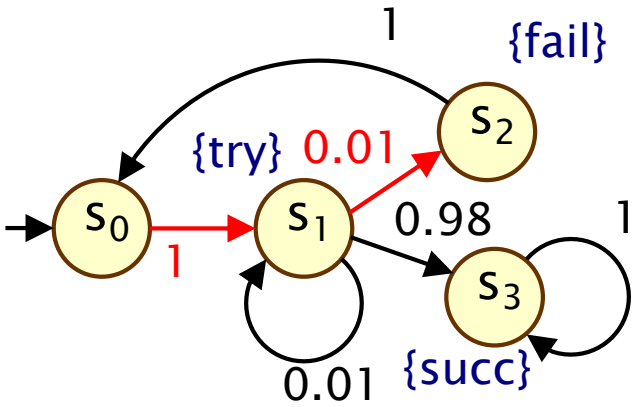
Probability space over paths

- Sample space Ω = Path(s)

  set of infinite paths with initial state s
- Event set $\Sigma_{Path(s)}$
  - the cylinder set $C(\omega) = \{ \omega' \in Path(s) \mid \omega \text{ is prefix of } \omega' \}$
  - $\Sigma_{Path(s)}$ is the least σ-algebra on Path(s) containing $C(\omega)$ for all finite paths ω starting in s
- Probability measure Pr_s
  - define probability $\mathbf{P}_s(\omega)$ for finite path $\omega = ss_1 \ldots s_n$ as:
    - $\mathbf{P}_s(\omega) = 1$ if ω has length one (i.e. $\omega = s$)
    - $\mathbf{P}_s(\omega) = \mathbf{P}(s,s_1) \cdot \ldots \cdot \mathbf{P}(s_{n-1},s_n)$ otherwise
    - define $Pr_s(C(\omega)) = \mathbf{P}_s(\omega)$ for all finite paths ω
  - Pr_s extends uniquely to a probability measure $Pr_s: \Sigma_{Path(s)} \rightarrow [0,1]$
- See [FKNP11] for further details

Probability space – Example

- Paths where sending fails the first time
  - $\omega = s_0 s_1 s_2$
  - $C(\omega)$ = all paths starting $s_0 s_1 s_2 \ldots$
  - $P_{s0}(\omega) = P(s_0,s_1) \cdot P(s_1,s_2)$
    $= 1 \cdot 0.01 = 0.01$
  - $Pr_{s0}(C(\omega)) = P_{s0}(\omega) = 0.01$



- Paths which are eventually successful and with no failures
  - $C(s_0 s_1 s_3) \cup C(s_0 s_1 s_1 s_3) \cup C(s_0 s_1 s_1 s_1 s_3) \cup \ldots$
  - $Pr_{s0}(C(s_0 s_1 s_3) \cup C(s_0 s_1 s_1 s_3) \cup C(s_0 s_1 s_1 s_1 s_3) \cup \ldots)$
    $= P_{s0}(s_0 s_1 s_3) + P_{s0}(s_0 s_1 s_1 s_3) + P_{s0}(s_0 s_1 s_1 s_1 s_3) + \ldots$
    $= 1 \cdot 0.98 + 1 \cdot 0.01 \cdot 0.98 + 1 \cdot 0.01 \cdot 0.01 \cdot 0.98 + \ldots$
    $= 0.9898989898\ldots$
    $= 98/99$

Reachability

- Key property: probabilistic reachability
  - probability of a path reaching a state in some target set $T \subseteq S$
  - e.g. “probability of the algorithm terminating successfully?”
  - e.g. “probability that an error occurs during execution?”

- Dual of reachability: invariance (safety)
  - probability of remaining within some class of states
  - Pr(“remain in set of states T”) = 1 – Pr(“reach set S\T”)
  - e.g. “probability that an error never occurs”

- We will also consider other variants of reachability
  - time-bounded, constrained (“until”), …

Reachability probabilities

- Formally: ProbReach(s, T) = Pr_s(Reach(s, T))
  - where Reach(s, T) = { $s_0 s_1 s_2 \ldots \in$ Path(s) | s_i in T for some i }

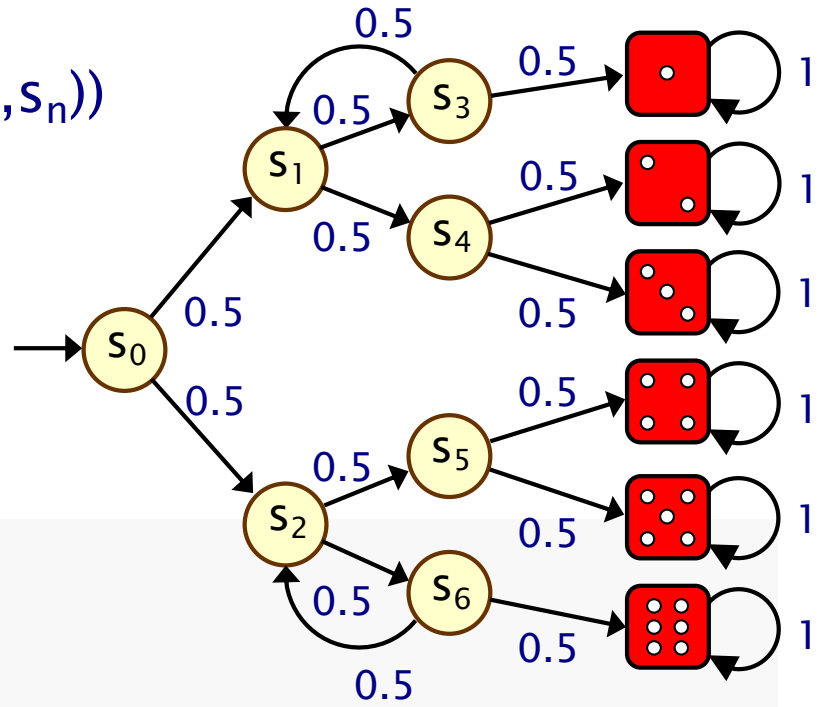
- Is Reach(s, T) measurable for any $T \subseteq S$? Yes…
  - Reach(s, T) is the union of all basic cylinders Cyl($s_0 s_1 \ldots s_n$) where $s_0 s_1 \ldots s_n$ in $Reach_{fin}$(s, T)
  - $Reach_{fin}$(s, T) contains all finite paths $s_0 s_1 \ldots s_n$ such that: $s_0 = s$, $s_0, \ldots, s_{n-1} \notin T$, $s_n \in T$ (reaches T first time)
  - set of such finite paths $s_0 s_1 \ldots s_n$ is countable

- Probability
  - in fact, the above is a disjoint union
  - so probability obtained by simply summing…

Computing reachability probabilities

- Compute as (infinite) sum…
- $\Sigma_{s_0,\ldots,s_n \in \text{Reachfin}(s,\,T)} \Pr_{s0}(\text{Cyl}(s_0,\ldots,s_n))$

  $= \Sigma_{s_0,\ldots,s_n \in \text{Reachfin}(s,\,T)} P(s_0,\ldots,s_n)$
- Example:
  - ProbReach(s_0, {4})

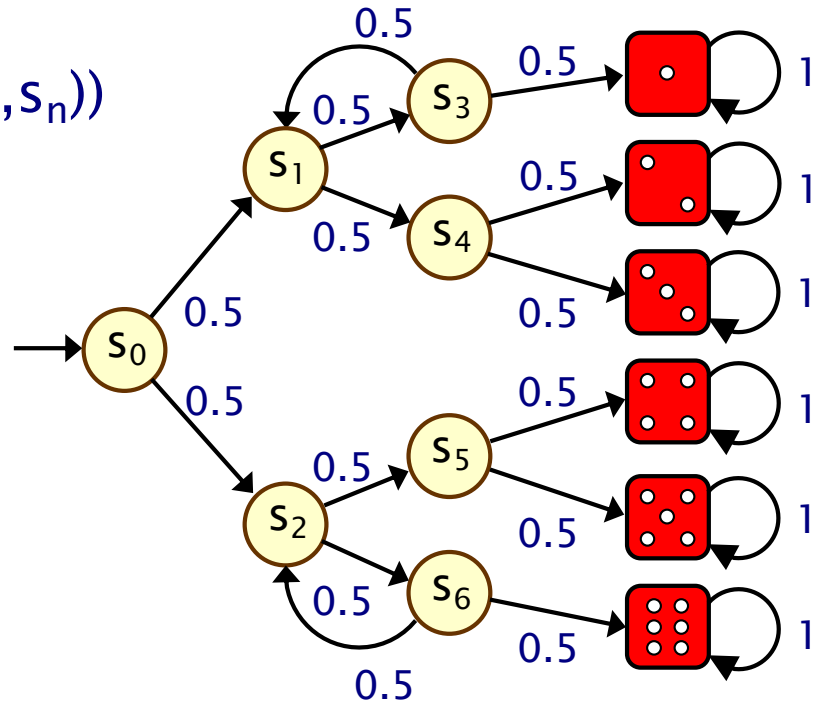


Computing reachability probabilities

- Compute as (infinite) sum…

- $\Sigma_{s_0,\ldots,s_n \in \text{Reachfin}(s,\,T)} \Pr_{s0}(\text{Cyl}(s_0,\ldots,s_n))$

  $= \Sigma_{s_0,\ldots,s_n \in \text{Reachfin}(s,\,T)} \mathbf{P}(s_0,\ldots,s_n)$



- Example:
  - ProbReach(s_0, {4})
  - = $\Pr_{s0}$(Reach(s_0, {4}))
  - Finite path fragments:
  - $s_0(s_2s_6)^n s_2s_5 4$ for $n \geq 0$
  - $\mathbf{P}_{s0}(s_0s_2s_5 4) + \mathbf{P}_{s0}(s_0s_2s_6s_2s_5 4) + \mathbf{P}_{s0}(s_0s_2s_6s_2s_6s_2s_5 4) + \ldots$
  - $= (1/2)^3 + (1/2)^5 + (1/2)^7 + \ldots = 1/6$

Computing reachability probabilities

- Alternative: derive a linear equation system
  - solve for all states simultaneously
  - i.e. compute vector ProbReach(T)

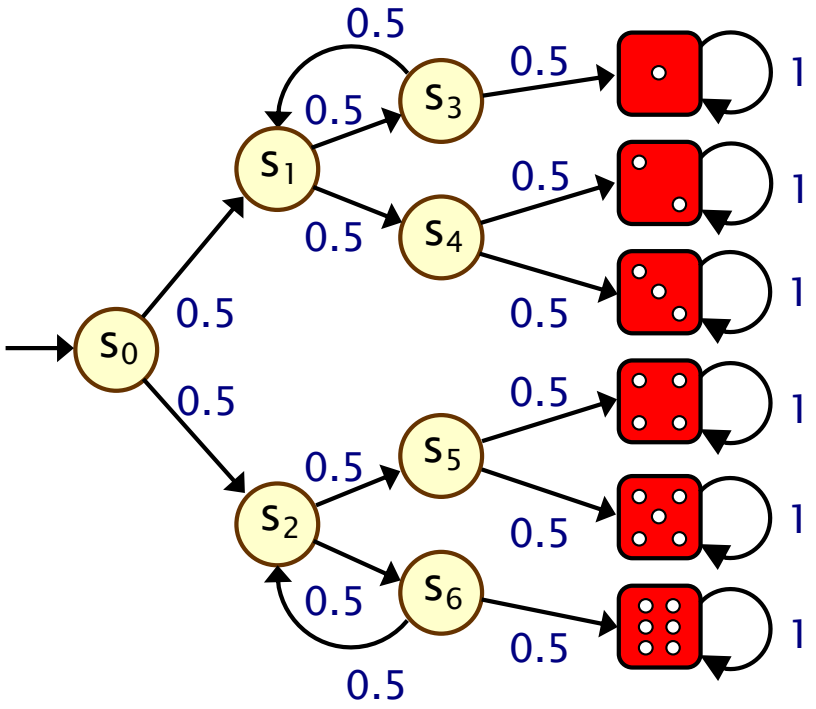
- Let x_s denote ProbReach(s, T)

- Solve:

$$x_s = \begin{cases} 1 & \text{if } s \in T \\ 0 & \text{if T is not reachable from s} \\ \sum_{s' \in S} \mathbf{P}(s,s') \cdot x_{s'} & \text{otherwise} \end{cases}$$

Example

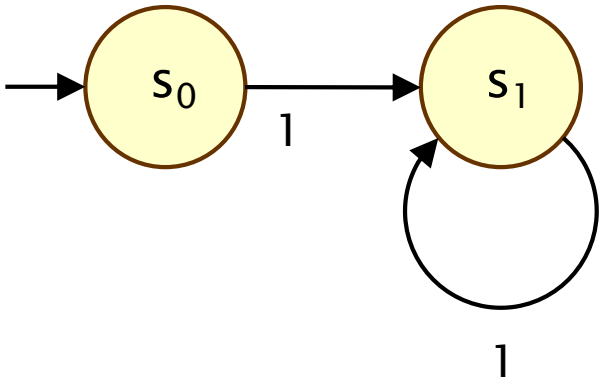
- Compute ProbReach(s_0, {4})



Unique solutions

- Why the need to identify states that cannot reach T?

- Consider this simple DTMC:
  - compute probability of reaching $\{s_0\}$ from s_1



  - linear equation system: $x_{s0} = 1$, $x_{s1} = x_{s1}$
  - multiple solutions: $(x_{s0}, x_{s1}) = (1,p)$ for any $p \in [0,1]$

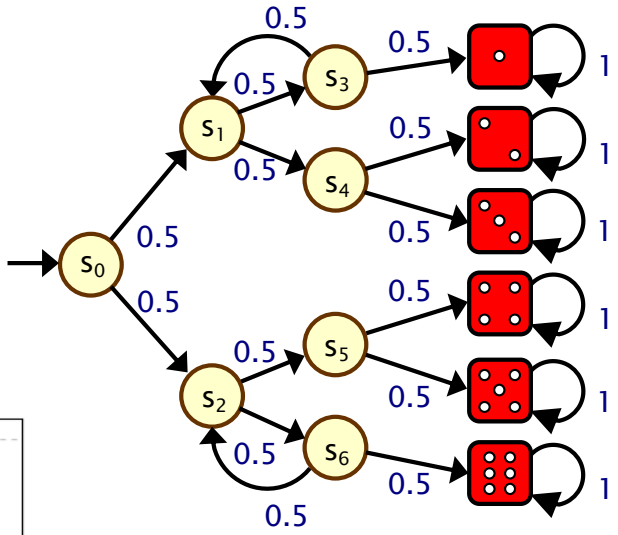
Bounded reachability probabilities

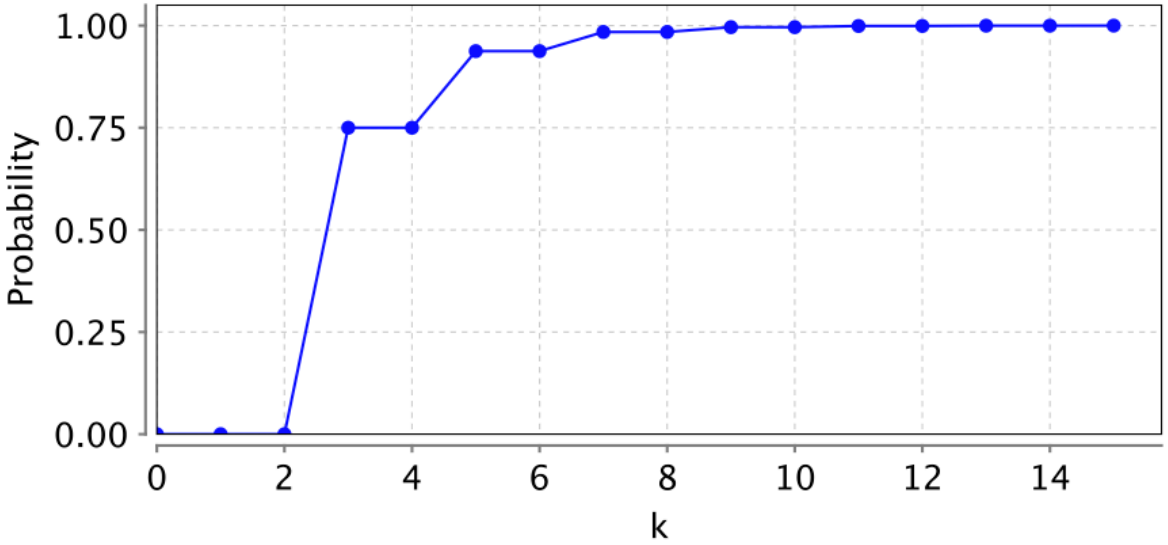
- Probability of reaching T from s within k steps
- Formally: $\text{ProbReach}^{\leq k}(s, T) = \Pr_s(\text{Reach}^{\leq k}(s, T))$ where:
  - $\text{Reach}^{\leq k}(s, T) = \{ s_0 s_1 s_2 \ldots \in \text{Path}(s) \mid s_i \text{ in } T \text{ for some } i \leq k \}$
- $\underline{\text{ProbReach}}^{\leq k}(T) = \underline{x}^{(k+1)}$ from the previous fixed point
  - which gives us…

$$\text{ProbReach}^{\leq k}(s, T) = \begin{cases} 1 & \text{if } s \in T \\ 0 & \text{if } k = 0 \,\&\, s \notin T \\ \sum_{s' \in S} \mathbf{P}(s,s') \cdot \text{ProbReach}^{\leq k-1}(s', T) & \text{if } k > 0 \,\&\, s \notin T \end{cases}$$

(Bounded) reachability

- ProbReach(s_0, {1,2,3,4,5,6}) = 1
- ProbReach$^{\leq k}$ (s_0, {1,2,3,4,5,6}) = …



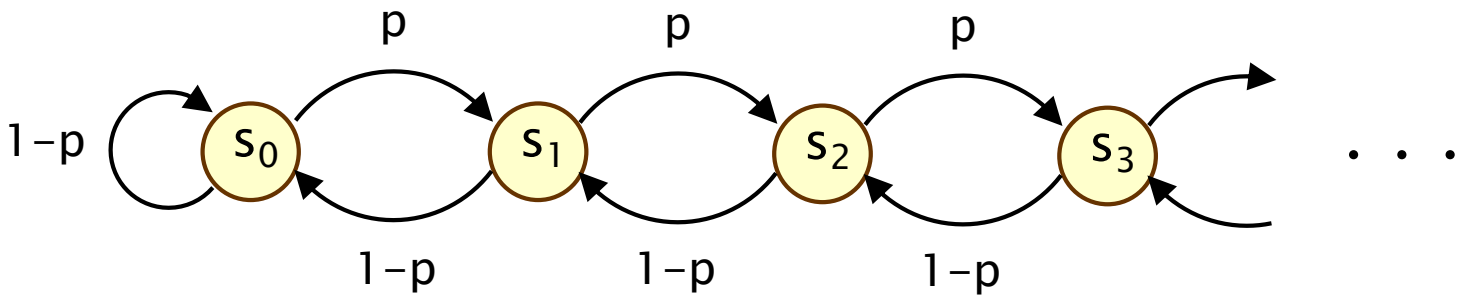


Qualitative properties

- Quantitative properties:
  - "what is the probability of event A?"
- Qualititative properties:
  - "the probability of event A is 1" ("almost surely A")
  - or: "the probability of event A is > 0" ("possibly A")
- For finite DTMCs, qualitative properties do not depend on the transition probabilities – only need underlying graph
  - e.g. to determine "is target set T reached with probability 1?" (see DTMC model checking later)

Aside: Infinite Markov chains

- Infinite-state random walk

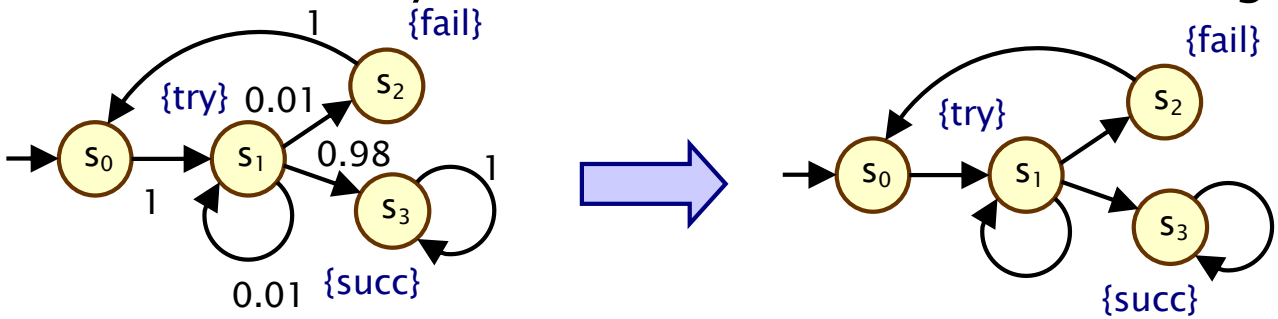


- Value of probability p does affect qualitative properties
  - ProbReach(s, $\{s_0\}$) = 1 if $p \leq 0.5$
  - ProbReach(s, $\{s_0\}$) < 1 if $p > 0.5$

Temporal logic

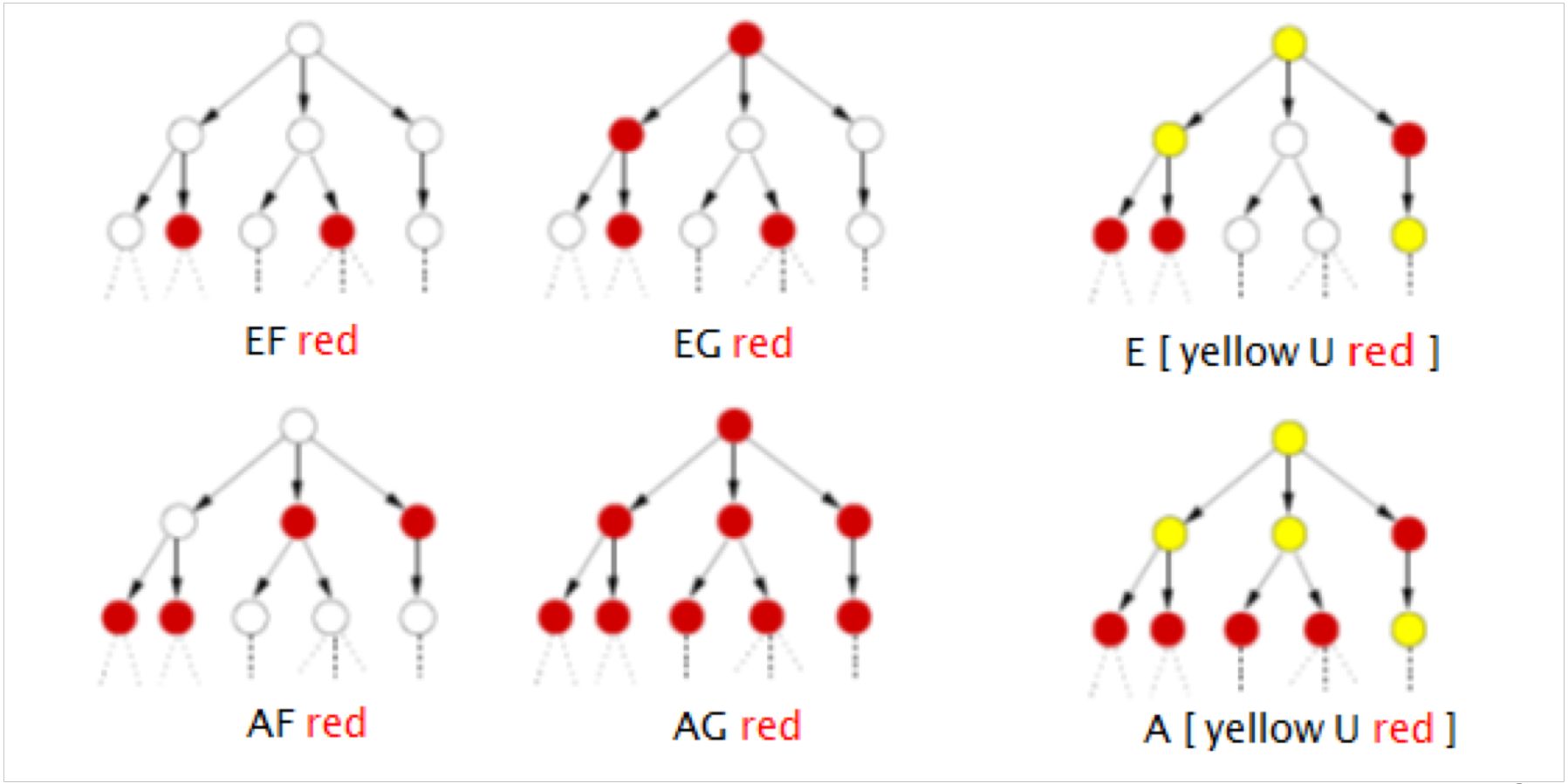
- Temporal logic
  - formal language for specifying and reasoning about how the behaviour of a system changes over time
  - defined over paths, i.e. sequences of states $s_0 s_1 s_2 s_3 \ldots$ such that $\mathbf{P}(s_i, s_{i+1}) > 0 \ \forall i$

- Logics used in this course are probabilistic extensions of temporal logics devised for non-probabilistic systems (CTL, LTL)
  - so we revert briefly to (labelled) state-transition diagrams



CTL semantics

- Intuitive semantics:
  - of quantifiers (A/E) and temporal operators (F/G/U)



PCTL

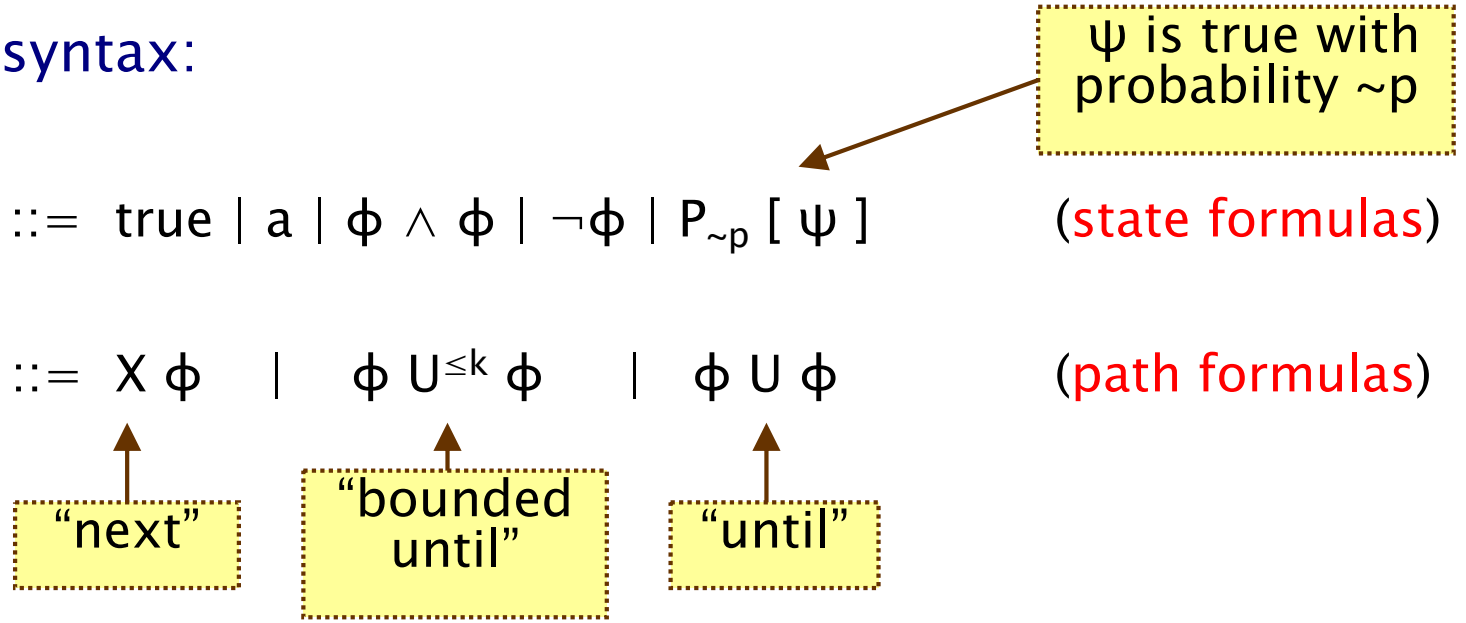
- Temporal logic for describing properties of DTMCs
  - PCTL = Probabilistic Computation Tree Logic [HJ94]
  - essentially the same as the logic pCTL of [ASB+95]

- Extension of (non-probabilistic) temporal logic CTL
  - key addition is probabilistic operator P
  - quantitative extension of CTL's A and E operators

- Example
  - send → $P_{\geq 0.95}$ [$F^{\leq 10}$ deliver]
  - "if a message is sent, then the probability of it being delivered within 10 steps is at least 0.95"

PCTL syntax

- PCTL syntax:



  - where a is an atomic proposition, used to identify states of interest, $p \in [0,1]$ is a probability, $\sim \in \{<,>,\leq,\geq\}$, $k \in \mathbb{N}$

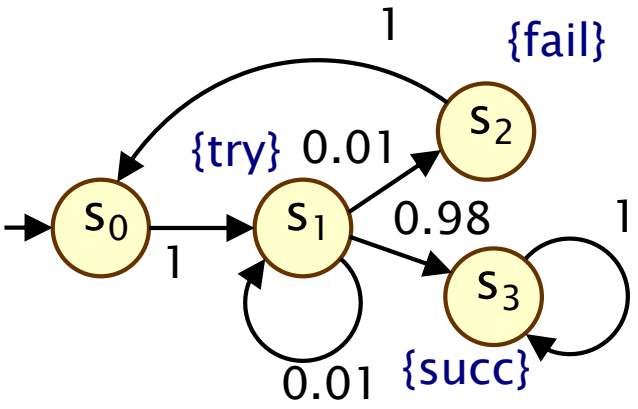
- A PCTL formula is always a state formula
  - path formulas only occur inside the P operator

PCTL semantics for DTMCs

- PCTL formulas interpreted over states of a DTMC
  - $s \models \phi$ denotes ϕ is "true in state s" or "satisfied in state s"

- Semantics of (non-probabilistic) state formulas:
  - for a state s of the DTMC $(S, s_{init}, \mathbf{P}, L)$:
  - $s \models a \quad \Leftrightarrow \quad a \in L(s)$
  - $s \models \phi_1 \wedge \phi_2 \quad \Leftrightarrow \quad s \models \phi_1$ and $s \models \phi_2$
  - $s \models \neg\phi \quad \Leftrightarrow \quad s \models \phi$ is false

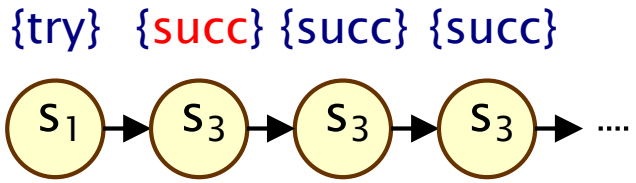
- Examples
  - $s_3 \models$ succ
  - $s_1 \models$ try $\wedge \neg$fail



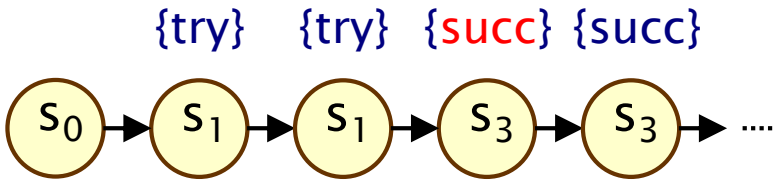
PCTL semantics for DTMCs

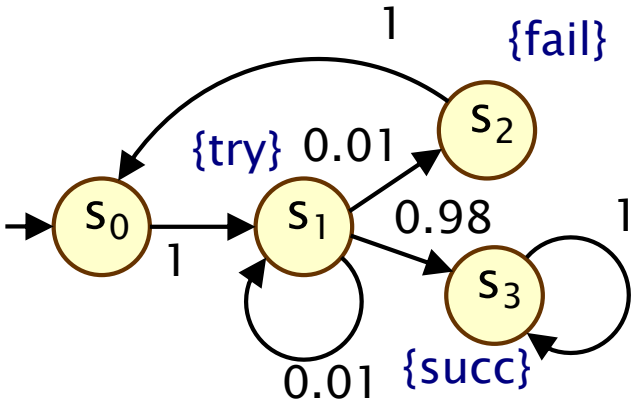
- Semantics of path formulas:
  - for a path $\omega = s_0 s_1 s_2 \ldots$ in the DTMC:
  - $\omega \models X\, \phi \quad \Leftrightarrow \quad s_1 \models \phi$
  - $\omega \models \phi_1\, U^{\leq k}\, \phi_2 \quad \Leftrightarrow \quad \exists i \leq k$ such that $s_i \models \phi_2$ and $\forall j < i$, $s_j \models \phi_1$
  - $\omega \models \phi_1\, U\, \phi_2 \quad \Leftrightarrow \quad \exists k \geq 0$ such that $\omega \models \phi_1\, U^{\leq k}\, \phi_2$

- Some examples of satisfying paths:
  - X succ



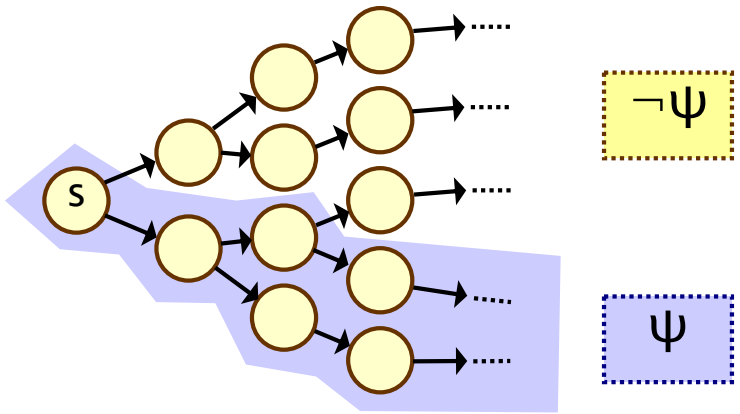
  - ¬fail U succ





PCTL semantics for DTMCs

- Semantics of the probabilistic operator P
  - informal definition: $s \models P_{\sim p}[\psi]$ means that "the probability, from state s, that ψ is true for an outgoing path satisfies $\sim p$"
  - example: $s \models P_{<0.25}[X\ fail] \Leftrightarrow$ "the probability of atomic proposition fail being true in the next state of outgoing paths from s is less than 0.25"
  - formally: $s \models P_{\sim p}[\psi] \Leftrightarrow Prob(s, \psi) \sim p$
  - where: $Prob(s, \psi) = Pr_s\{\omega \in Path(s) \mid \omega \models \psi\}$
  - (sets of paths satisfying ψ are always measurable [Var85])



$Prob(s, \psi) \sim p$?

More PCTL…

- Usual temporal logic equivalences:
  - false $\equiv \neg$true (false)
  - $\phi_1 \vee \phi_2 \equiv \neg(\neg\phi_1 \wedge \neg\phi_2)$ (disjunction)
  - $\phi_1 \rightarrow \phi_2 \equiv \neg\phi_1 \vee \phi_2$ (implication)

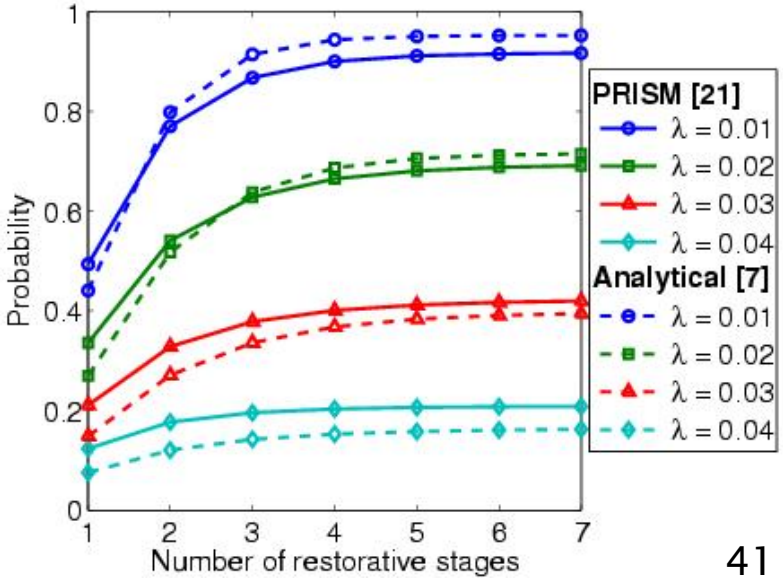
  - $F\ \phi \equiv \Diamond\ \phi \equiv$ true $U\ \phi$ (eventually, “future”)
  - $G\ \phi \equiv \Box\ \phi \equiv \neg(F\ \neg\phi)$ (always, “globally”)
  - bounded variants: $F^{\leq k}\ \phi$, $G^{\leq k}\ \phi$

- Negation and probabilities
  - e.g. $\neg P_{>p}\ [\ \phi_1\ U\ \phi_2\ ] \equiv P_{\leq p}\ [\phi_1\ U\ \phi_2\ ]$
  - e.g. $P_{>p}\ [\ G\ \phi\ ] \equiv P_{<1-p}\ [\ F\ \neg\phi\ ]$

Quantitative properties

- Consider a PCTL formula $P_{\sim p}$ [ψ]
  - if the probability is unknown, how to choose the bound p?
- When the outermost operator of a PTCL formula is P
  - we allow the form $P_{=?}$ [ψ]
  - "what is the probability that path formula ψ is true?"
- Model checking is no harder: compute the values anyway
- Useful to spot patterns, trends

- Example
  - $P_{=?}$ [F err/total>0.1]
  - "what is the probability that 10% of the NAND gate outputs are erroneous?"



Reachability and invariance

- Derived temporal operators, like CTL…

- Probabilistic reachability: $P_{\sim p}$ [F ϕ]
  - the probability of reaching a state satisfying ϕ
  - F $\phi \equiv$ true U ϕ
  - "ϕ is eventually true"
  - bounded version: $F^{\leq k}\ \phi \equiv$ true $U^{\leq k}\ \phi$

- Probabilistic invariance: $P_{\sim p}$ [G ϕ]
  - the probability of ϕ always remaining true
  - G $\phi \equiv \neg$(F $\neg\phi$) $\equiv \neg$(true U $\neg\phi$)
  - "ϕ is always true"
  - bounded version: $G^{\leq k}\ \phi \equiv \neg(F^{\leq k}\ \neg\phi)$

strictly speaking, G ϕ cannot be derived from the PCTL syntax in this way since there is no negation of path formulae

Qualitative vs. quantitative properties

- P operator of PCTL can be seen as a quantitative analogue of the CTL operators A (for all) and E (there exists)

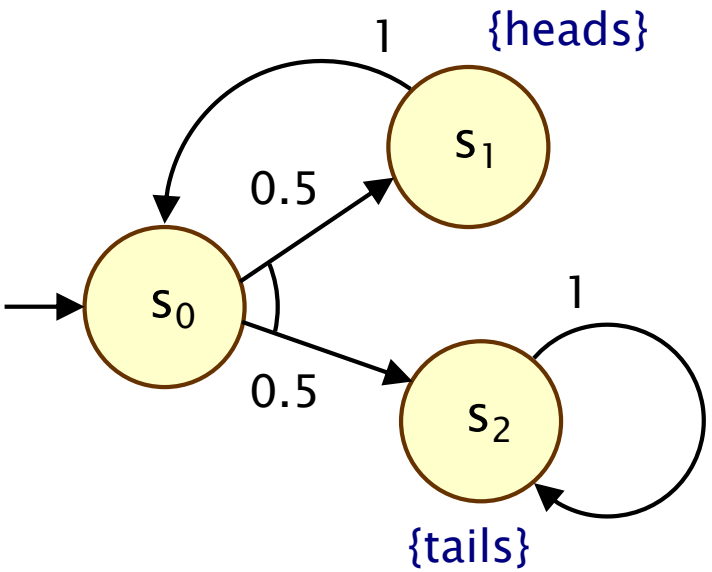
- Qualitative PCTL properties
  - $P_{\sim p}$ [ψ] where p is either 0 or 1
- Quantitative PCTL properties
  - $P_{\sim p}$ [ψ] where p is in the range (0,1)

- $P_{>0}$ [F ϕ] is identical to EF ϕ
  - there exists a finite path to a ϕ-state
- $P_{\geq 1}$ [F ϕ] is (similar to but) weaker than AF ϕ
  - a ϕ-state is reached "almost surely"
  - see next slide…

Example: Qualitative/quantitative

- Toss a coin repeatedly until “tails” is thrown

- Is “tails” always eventually thrown?
  - CTL: AF “tails”
  - Result: false
  - Counterexample: $s_0 s_1 s_0 s_1 s_0 s_1 \ldots$



- Does the probability of eventually throwing “tails” equal one?
  - PCTL: $P_{\geq 1}$ [F “tails”]
  - Result: true
  - Infinite path $s_0 s_1 s_0 s_1 s_0 s_1 \ldots$ has zero probability

Overview (Part 1)

- Probability basics
- Discrete-time Markov chains (DTMCs)
  - definition, paths & probability spaces
- Temporal logic PCTL
- PCTL model checking
- Costs and rewards
- PRISM: overview
  - Modelling language
  - Properties
  - GUI, etc
  - Case study: Bluetooth device discovery
- Summary

PCTL model checking for DTMCs

- Algorithm for PCTL model checking [CY88,HJ94,CY95]
  - inputs: DTMC $D=(S,s_{init},\mathbf{P},L)$, PCTL formula ϕ
  - output: $Sat(\phi) = \{ s \in S \mid s \models \phi \}$ = set of states satisfying ϕ

- What does it mean for a DTMC D to satisfy a formula ϕ?
  - sometimes, want to check that $s \models \phi \ \forall \ s \in S$, i.e. $Sat(\phi) = S$
  - sometimes, just want to know if $s_{init} \models \phi$, i.e. if $s_{init} \in Sat(\phi)$

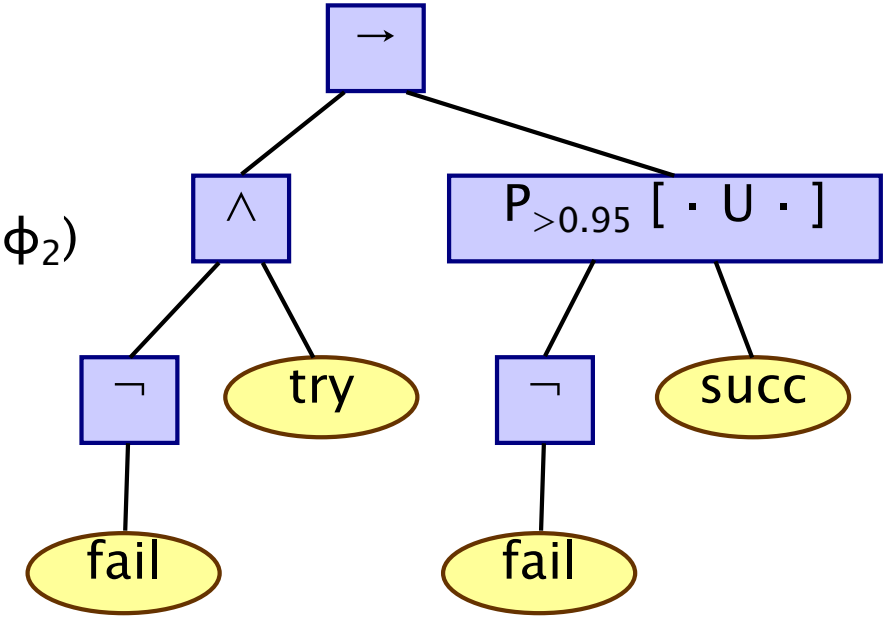
- Sometimes, focus on quantitative results
  - e.g. compute result of P=? [F error]
  - e.g. compute result of P=? [$F^{\leq k}$ error] for $0 \leq k \leq 100$

PCTL model checking for DTMCs

- Basic algorithm proceeds by induction on parse tree of ϕ
  - example: $\phi = (\neg \text{fail} \wedge \text{try}) \rightarrow P_{>0.95}$ [$\neg$fail U succ]

- For the non-probabilistic operators:
  - Sat(true) = S
  - Sat(a) = { $s \in S \mid a \in L(s)$ }
  - $\text{Sat}(\neg\phi) = S \setminus \text{Sat}(\phi)$
  - $\text{Sat}(\phi_1 \wedge \phi_2) = \text{Sat}(\phi_1) \cap \text{Sat}(\phi_2)$

- For the $P_{\sim p}$ [ψ] operator
  - need to compute the probabilities Prob(s, ψ) for all states $s \in S$



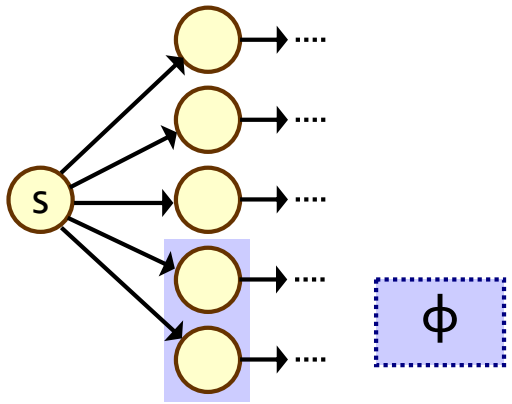
Probability computation

- Three temporal operators to consider:
- Next: $P_{\sim p}[\ X\ \phi\ ]$
- Bounded until: $P_{\sim p}[\ \phi_1\ U^{\leq k}\ \phi_2\ ]$ (omitted)
  - adaptation of bounded reachability for DTMCs
- Until: $P_{\sim p}[\ \phi_1\ U\ \phi_2\ ]$
  - adaptation of reachability for DTMCs
  - graph-based "precomputation" algorithms
  - techniques for solving large linear equation systems

PCTL next for DTMCs

- Computation of probabilities for PCTL next operator
  - $Sat(P_{\sim p}[\ X\ \phi\ ]) = \{\ s \in S \mid Prob(s, X\ \phi) \sim p\ \}$
  - need to compute $Prob(s, X\ \phi)$ for all $s \in S$

- Sum outgoing probabilities for transitions to ϕ-states
  - $Prob(s, X\ \phi) = \Sigma_{s' \in Sat(\phi)}\ \mathbf{P}(s,s')$

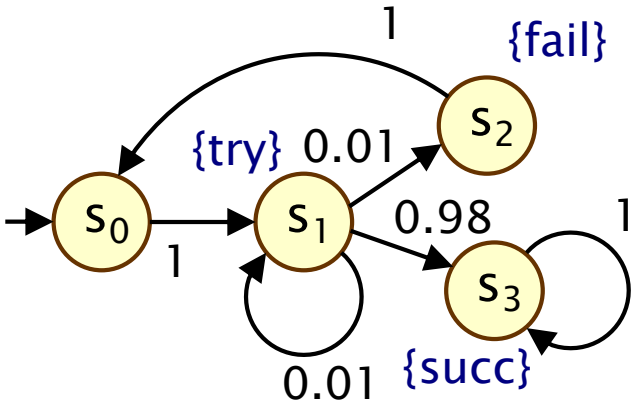


- Compute vector $\underline{Prob}(X\ \phi)$ of probabilities for all states s
  - $\underline{Prob}(X\ \phi) = \mathbf{P} \cdot \underline{\phi}$
  - where $\underline{\phi}$ is a 0-1 vector over S with $\underline{\phi}(s) = 1$ iff $s \models \phi$
  - computation requires a single matrix-vector multiplication

PCTL next – Example

- Model check: $P_{\geq 0.9}$ [X (¬try ∨ succ)]
  - Sat (¬try ∨ succ) = (S \ Sat(try)) ∪ Sat(succ)
    = ({s_0,s_1,s_2,s_3} \ {s_1}) ∪ {s_3} = {s_0,s_2,s_3}
  - Prob(X (¬try ∨ succ)) = **P** · (¬try ∨ succ) = …

$$= \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0.01 & 0.01 & 0.98 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.99 \\ 1 \\ 1 \end{bmatrix}$$



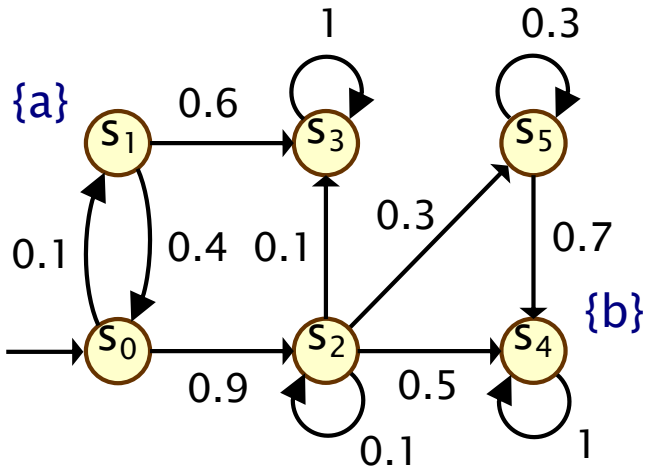
- Results:
  - Prob(X (¬try ∨ succ)) = [0, 0.99, 1, 1]
  - Sat($P_{\geq 0.9}$ [X (¬try ∨ succ)]) = {s_1, s_2, s_3}

PCTL until for DTMCs

- Computation of probabilities Prob(s, ϕ_1 U ϕ_2) for all s $\in$ S
- First, identify all states where the probability is 1 or 0
  - $S^{yes} = Sat(P_{\geq 1}[\ \phi_1\ U\ \phi_2\ ])$
  - $S^{no} = Sat(P_{\leq 0}[\ \phi_1\ U\ \phi_2\ ])$
- Then solve linear equation system for remaining states

- Running example:

$P_{>0.8}[\neg a\ U\ b\ ]$



Precomputation

- We refer to the first phase (identifying sets S^{yes} and S^{no}) as "precomputation"
  - two algorithms: Prob0 (for S^{no}) and Prob1 (for S^{yes})
  - algorithms work on underlying graph (probabilities irrelevant)

- Important for several reasons
  - ensures unique solution to linear equation system
    - only need Prob0 for uniqueness, Prob1 is optional
  - reduces the set of states for which probabilities must be computed numerically
  - gives exact results for the states in S^{yes} and S^{no} (no round-off)
  - for model checking of qualitative properties ($P_{\sim p}[\cdot]$ where p is 0 or 1), no further computation required

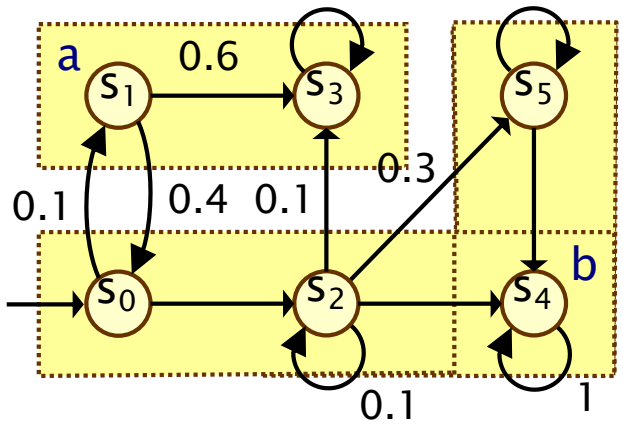
Precomputation – Prob0

- Prob0 algorithm to compute $S^{no} = Sat(P_{\leq 0}[\ \phi_1\ U\ \phi_2\ ])$:
  - first compute $Sat(P_{>0}[\ \phi_1\ U\ \phi_2\ ]) \equiv Sat(E[\ \phi_1\ U\ \phi_2\ ])$
  - i.e. find all states which can, with non-zero probability, reach a ϕ_2-state without leaving ϕ_1-states
  - i.e. find all states from which there is a finite path through ϕ_1-states to a ϕ_2-state: simple graph-based computation
  - subtract the resulting set from S

$S^{no} = Sat(P_{\leq 0}[\neg a\ U\ b\ ])$

Example:

$P_{>0.8}[\neg a\ U\ b\ ]$



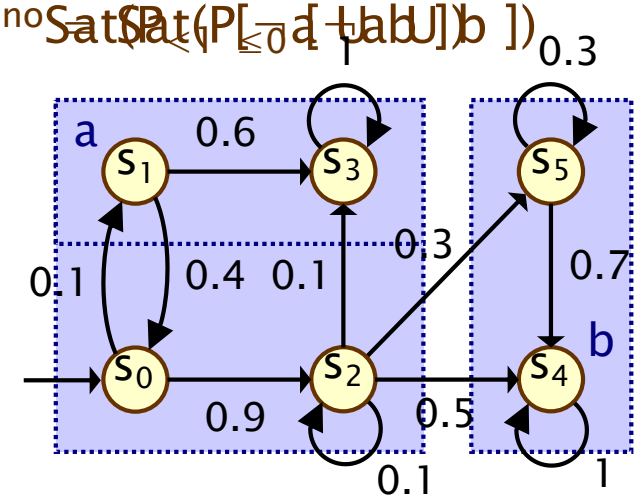
$Sat(P_{>0}[\neg a\ U\ b\ ])$

Precomputation – Prob1

- Prob1 algorithm to compute $S^{yes} = Sat(P_{\geq 1}\ [\ \phi_1\ U\ \phi_2\ ])$:
  - first compute $Sat(P_{<1}\ [\ \phi_1\ U\ \phi_2\ ])$, reusing S^{no}
  - this is equivalent to the set of states which have a non-zero probability of reaching S^{no}, passing only through ϕ_1-states
  - again, this is a simple graph-based computation
  - subtract the resulting set from S

Example:

$P_{>0.8}\ [\neg a\ U\ b\ ]$



$S^{yes} =$

$Sat(P_{\geq 1}\ [\neg a\ U\ b\ ])$

PCTL until – linear equations

- Probabilities Prob(s, ϕ_1 U ϕ_2) can now be obtained as the unique solution of the following set of linear equations
  - essentially the same as for probabilistic reachability

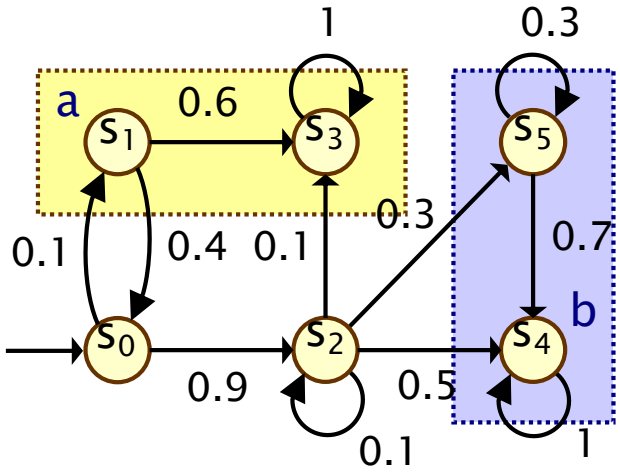
$$\text{Prob}(s,\ \phi_1\ \text{U}\ \phi_2) \quad = \quad \begin{cases} 1 & \text{if } s \in S^{yes} \\ 0 & \text{if } s \in S^{no} \\ \sum_{s' \in S} P(s,s') \cdot \text{Prob}(s',\ \phi_1\ \text{U}\ \phi_2) & \text{otherwise} \end{cases}$$

- Can also be reduced to a system in $|S^?|$ unknowns instead of $|S|$ where $S^? = S \setminus (S^{yes} \cup S^{no})$

PCTL until – linear equations

- Example: $P_{>0.8}$ [¬a U b]
- Let $x_i = Prob(s_i, \neg a \ U \ b)$

$S^{no} =$ Sat($P_{\leq 0}$ [¬a U b])



$S^{yes} =$ Sat($P_{\geq 1}$ [¬a U b])

$x_1 = x_3 = 0$

$x_4 = x_5 = 1$

$x_2 = 0.1x_2+0.1x_3+0.3x_5+0.5x_4 = 8/9$

$x_0 = 0.1x_1+0.9x_2 = 0.8$

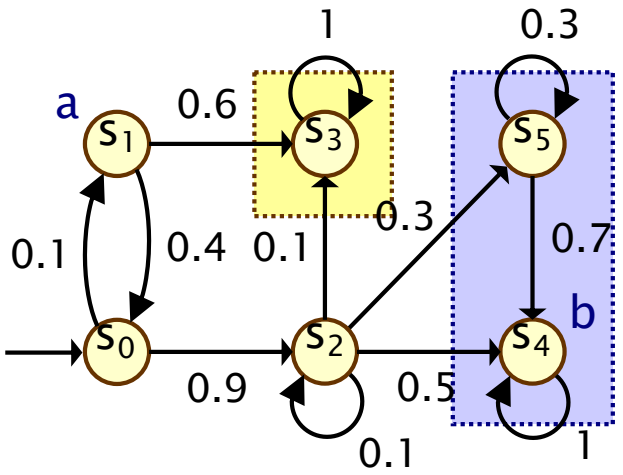
$\underline{Prob}(\neg a \ U \ b) = \underline{x} = [0.8, 0, 8/9, 0, 1, 1]$

$Sat(P_{>0.8}$ [¬a U b]) = { s_2, s_4, s_5 }

PCTL Until – Example 2

- Example: $P_{>0.5}$ [G¬b]
- Prob(s_i, G¬b)
  = 1 – Prob(s_i, ¬(G¬b))
  = 1 – Prob(s_i, F b)
- Let x_i = Prob(s_i, F b)

$S^{no} = Sat(P_{\leq 0}$ [F b])



S^{yes} = Sat($P_{\geq 1}$ [F b])

$x_3 = 0$ and $x_4 = x_5 = 1$

$x_2 = 0.1x_2+0.1x_3+0.3x_5+0.5x_4 = 8/9$

$x_1 = 0.6x_3+0.4x_0 = 0.4x_0$

$x_0 = 0.1x_1+0.9x_2 = 5/6$ and $x_1 = 1/3$

$\underline{Prob}(G\neg b) = \underline{1}-\underline{x} = [1/6, 2/3, 1/9, 1, 0, 0]$

$Sat(P_{>0.5}$ [G¬b]) = { s_1,s_3 }

Linear equation systems

- Solution of large (sparse) linear equation systems
  - size of system (number of variables) typically O(|S|)
  - state space S gets very large in practice
- Two main classes of solution methods:
  - direct methods – compute exact solutions in fixed number of steps, e.g. Gaussian elimination, L/U decomposition
  - iterative methods, e.g. Power, Jacobi, Gauss-Seidel, …
  - the latter are preferred in practice due to scalability

- General form: $\mathbf{A} \cdot \underline{x} = \underline{b}$
  - indexed over integers,
  - i.e. assume S = { 0,1,…,|S|-1 }

$$\sum_{j=0}^{|S|-1} \mathbf{A}(i,j) \cdot \underline{x}(j) \quad = \quad \underline{b}(i)$$

Limitations of PCTL

- PCTL, although useful in practice, has limited expressivity
  - essentially: probability of reaching states in X, passing only through states in Y (and within k time-steps)

- More expressive logics can be used, for example:
  - LTL [Pnu77] – (non-probabilistic) linear-time temporal logic
  - PCTL* [ASB+95,BdA95] – which subsumes both PCTL and LTL
  - both allow path operators to be combined
  - (in PCTL, $P_{\sim p}$ [...] always contains a single temporal operator)
  - supported by PRISM
  - (not covered in this lecture)

- Another direction: extend DTMCs with costs and rewards...

Costs and rewards

- We augment DTMCs with rewards (or, conversely, costs)
  - real-valued quantities assigned to states and/or transitions
  - these can have a wide range of possible interpretations
- Some examples:
  - elapsed time, power consumption, size of message queue, number of messages successfully delivered, net profit, …
- Costs? or rewards?
  - mathematically, no distinction between rewards and costs
  - when interpreted, we assume that it is desirable to minimise costs and to maximise rewards
  - we will consistently use the terminology “rewards” regardless

Reward-based properties

- Properties of DTMCs augmented with rewards
  - allow a wide range of quantitative measures of the system
  - basic notion: expected value of rewards
  - formal property specifications will be in an extension of PCTL

- More precisely, we use two distinct classes of property…

- Instantaneous properties
  - the expected value of the reward at some time point

- Cumulative properties
  - the expected cumulated reward over some period

DTMC reward structures

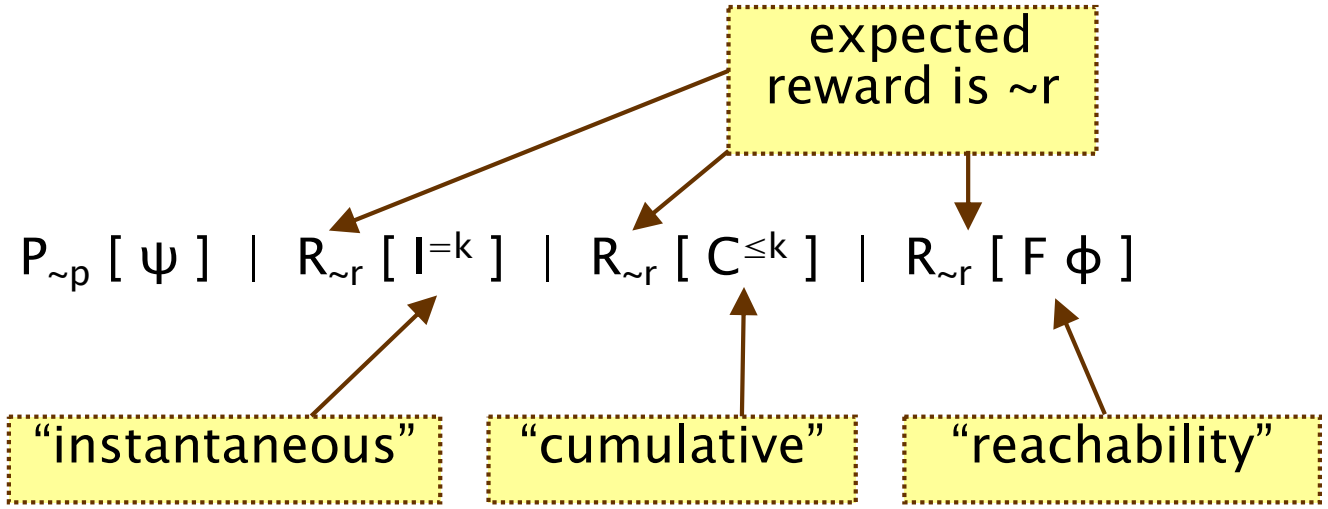
- For a DTMC $(S, s_{init}, \mathbf{P}, L)$, a reward structure is a pair $(\underline{\rho}, \iota)$
  - $\underline{\rho} : S \to \mathbb{R}_{\geq 0}$ is the state reward function (vector)
  - $\iota : S \times S \to \mathbb{R}_{\geq 0}$ is the transition reward function (matrix)

- Example (for use with instantaneous properties)
  - "size of message queue": $\underline{\rho}$ maps each state to the number of jobs in the queue in that state, ι is not used

- Examples (for use with cumulative properties)
  - "time-steps": $\underline{\rho}$ returns 1 for all states and ι is zero (equivalently, $\underline{\rho}$ is zero and ι returns 1 for all transitions)
  - "number of messages lost": $\underline{\rho}$ is zero and ι maps transitions corresponding to a message loss to 1
  - "power consumption": $\underline{\rho}$ is defined as the per-time-step energy consumption in each state and ι as the energy cost of each transition

PCTL and rewards

- Extend PCTL to incorporate reward-based properties
  - add an R operator, which is similar to the existing P operator
  - $\phi ::= \ldots \mid P_{\sim p}[\psi] \mid R_{\sim r}[I^{=k}] \mid R_{\sim r}[C^{\leq k}] \mid R_{\sim r}[F\,\phi]$



  - where $r \in \mathbb{R}_{\geq 0}$, $\sim \in \{<, >, \leq, \geq\}$, $k \in \mathbb{N}$

- $R_{\sim r}[\,\cdot\,]$ means "the expected value of $\cdot$ satisfies ~r"

Reward formula semantics

- Formal semantics of the three reward operators
  - based on random variables over (infinite) paths

- Recall:
  - $s \models P_{\sim p} [\ \psi\ ] \Leftrightarrow \Pr_s \{\ \omega \in \mathrm{Path}(s) \mid \omega \models \psi\ \} \sim p$

- For a state s in the DTMC (see [KNP07a] for full definition):
  - $s \models R_{\sim r} [\ I^{=k}\ ] \Leftrightarrow \mathrm{Exp}(s, X_{I=k}) \sim r$ (instantaenous)
  - $s \models R_{\sim r} [\ C^{\leq k}\ ] \Leftrightarrow \mathrm{Exp}(s, X_{C\leq k}) \sim r$ (cumulative)
  - $s \models R_{\sim r} [\ F\ \Phi\ ] \Leftrightarrow \mathrm{Exp}(s, X_{F\Phi}) \sim r$ (reachability)

  where: Exp(s, X) denotes the expectation of the random variable $X : \mathrm{Path}(s) \rightarrow \mathbb{R}_{\geq 0}$ with respect to the probability measure $\Pr_s$

Reward formula semantics

- Definition of random variables:
  - for an infinite path $\omega = s_0 s_1 s_2 \ldots$

$$X_{I=k}(\omega) = \underline{\rho}(s_k)$$

$$X_{C\leq k}(\omega) = \begin{cases} 0 & \text{if } k = 0 \\ \sum_{i=0}^{k-1} \underline{\rho}(s_i) + \iota(s_i, s_{i+1}) & \text{otherwise} \end{cases}$$

$$X_{F\phi}(\omega) = \begin{cases} 0 & \text{if } s_0 \in \mathrm{Sat}(\phi) \\ \infty & \text{if } s_i \notin \mathrm{Sat}(\phi) \text{ for all } i \geq 0 \\ \sum_{i=0}^{k_\phi - 1} \underline{\rho}(s_i) + \iota(s_i, s_{i+1}) & \text{otherwise} \end{cases}$$

  - where $k_\phi = \min\{ j \mid s_j \models \phi \}$

PRISM

- PRISM: Probabilistic symbolic model checker
  - developed at Birmingham/Oxford University, since 1999
  - free, open source software (GPL), runs on all major OSs



- Construction/analysis of probabilistic models…
  - discrete-time Markov chains, continuous-time Markov chains, Markov decision processes, probabilistic timed automata, stochastic multi-player games, …
- Simple but flexible high-level modelling language
  - based on guarded commands; see later…
- Many import/export options, tool connections
  - in: (Bio)PEPA, stochastic π-calculus, DSD, SBML, Petri nets, …
  - out: Matlab, MRMC, INFAMY, PARAM, …

PRISM…



- Model checking for various temporal logics…
  - PCTL, CSL, LTL, PCTL*, rPATL, CTL, …
  - quantitative extensions, costs/rewards, …
- Various efficient model checking engines and techniques
  - symbolic methods (binary decision diagrams and extensions)
  - explicit-state methods (sparse matrices, etc.)
  - statistical model checking (simulation-based approximations)
  - and more: symmetry reduction, quantitative abstraction refinement, fast adaptive uniformisation, …
- Graphical user interface
  - editors, simulator, experiments, graph plotting
- See: http://www.prismmodelchecker.org/
  - downloads, tutorials, case studies, papers, …

PRISM modelling language

- Simple, textual, state-based modelling language
  - used for all probabilistic models supported by PRISM
  - based on Reactive Modules [AH99]
- Language basics
  - system built as parallel composition of interacting modules
  - state of each module given by finite-ranging variables
  - behaviour of each module specified by guarded commands
    - annotated with probabilities/rates and (optional) action label
  - transitions are associated with state-dependent probabilities
  - interactions between modules through synchronisation

[send] (s=2) -> p_{loss} : (s'=3)&(lost'=lost+1) + (1-p_{loss}) : (s'=4);

action guard probability update probability update

Simple example

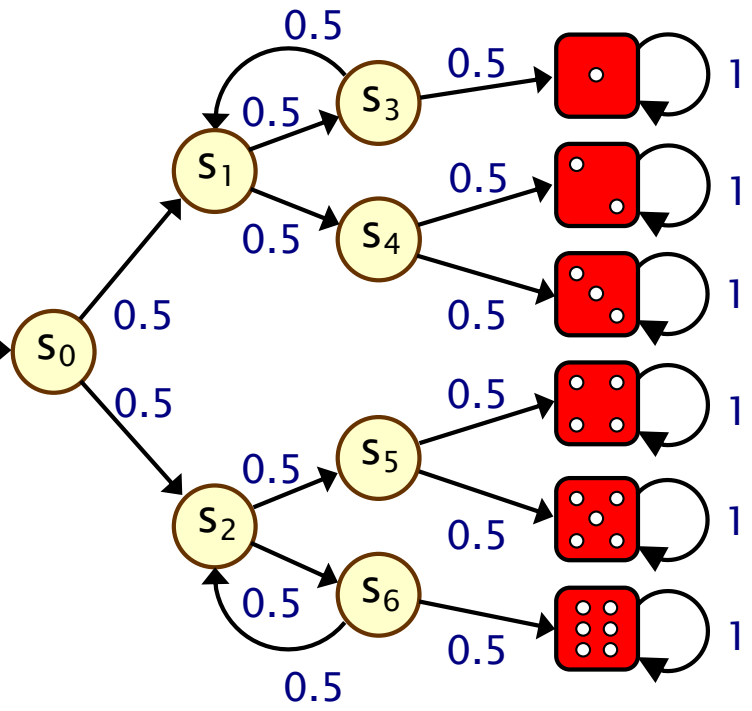
```
dtmc

module M1
    x : [0..3] init 0;
    [a] x=0 -> (x'=1);
    [b] x=1 -> 0.5 : (x'=2) + 0.5 : (x'=3);
endmodule

module M2
    y : [0..3] init 0;
    [a] y=0 -> (y'=1);
    [b] y=1 -> 0.4 : (y'=2) + 0.6 : (y'=3);
endmodule
```

Probabilistic models

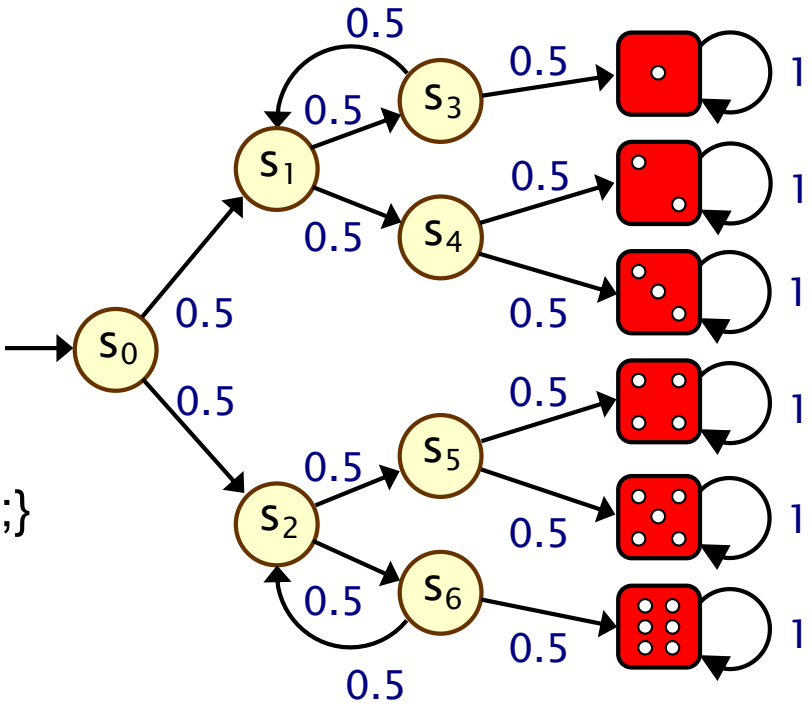
```
dtmc
module die
// local state s : [0..7] init 0;
// value of the dice d : [0..6] init 0;
[] s=0 -> 0.5 : (s'=1) + 0.5 : (s'=2);
…
[] s=3 ->
   0.5 : (s'=1) + 0.5 : (s'=7) & (d'=1);
[] s=4 ->
   0.5 : (s'=7) & (d'=2) + 0.5 : (s'=7) & (d'=3);
…
[] s=7 -> (s'=7);
endmodule
rewards "coin_flips"
[] s<7 : 1;
endrewards
```



Given in PRISM's guarded commands modelling notation

Probabilistic models

```
int s, d;
s = 0; d = 0;
while (s < 7) {
   bool coin = Bernoulli(0.5);
   if (s = 0)
      if (coin) s = 1 else s = 2;
...
  else if (s = 3)
      if (coin) s = 1 else {s = 7; d = 1;}
  else if (s = 4)
      if (coin) {s = 7; d = 2} else {s = 7; d = 3;}
…
}
return (d)
```



Given as a probabilistic program

Rewards in the PRISM language

```
rewards "total_queue_size"
    true : queue1+queue2;
endrewards
```

(instantaneous, state rewards)

```
rewards "time"
    true : 1;
endrewards
```

(cumulative, state rewards)

```
rewards "dropped"
    [receive] q=q_max : 1;
endrewards
```

(cumulative, transition rewards)
(q = queue size, q_max = max. queue size, receive = action label)

```
rewards "power"
    sleep=true : 0.25;
    sleep=false : 1.2 * up;
    [wake] true : 3.2;
endrewards
```

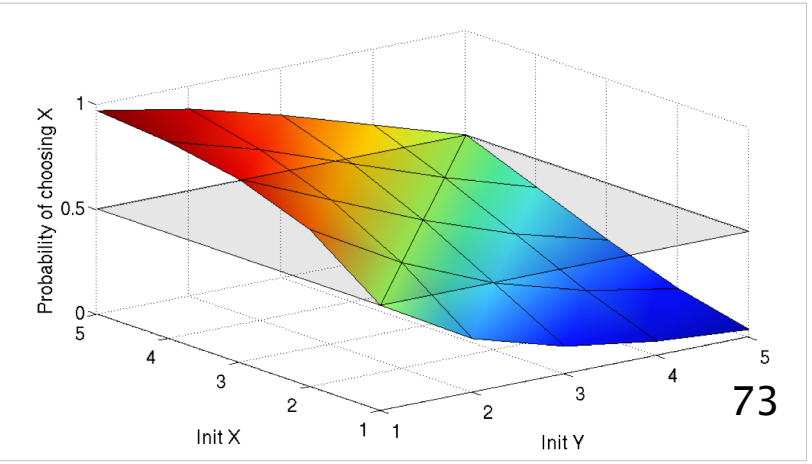
(cumulative, state/trans. rewards)
(up = num. operational components, wake = action label)

PRISM – Property specification

- Temporal logic property specification language
  - subsumes PCTL, CSL, probabilistic LTL, PCTL*, …

- Simple examples:
  - $P_{\leq 0.01}$ [F "crash"] – "the probability of a crash is at most 0.01"
  - $S_{>0.999}$ ["up"] – "long-run probability of availability is >0.999"
  - note: P takes a path formula as argument, S a state formula

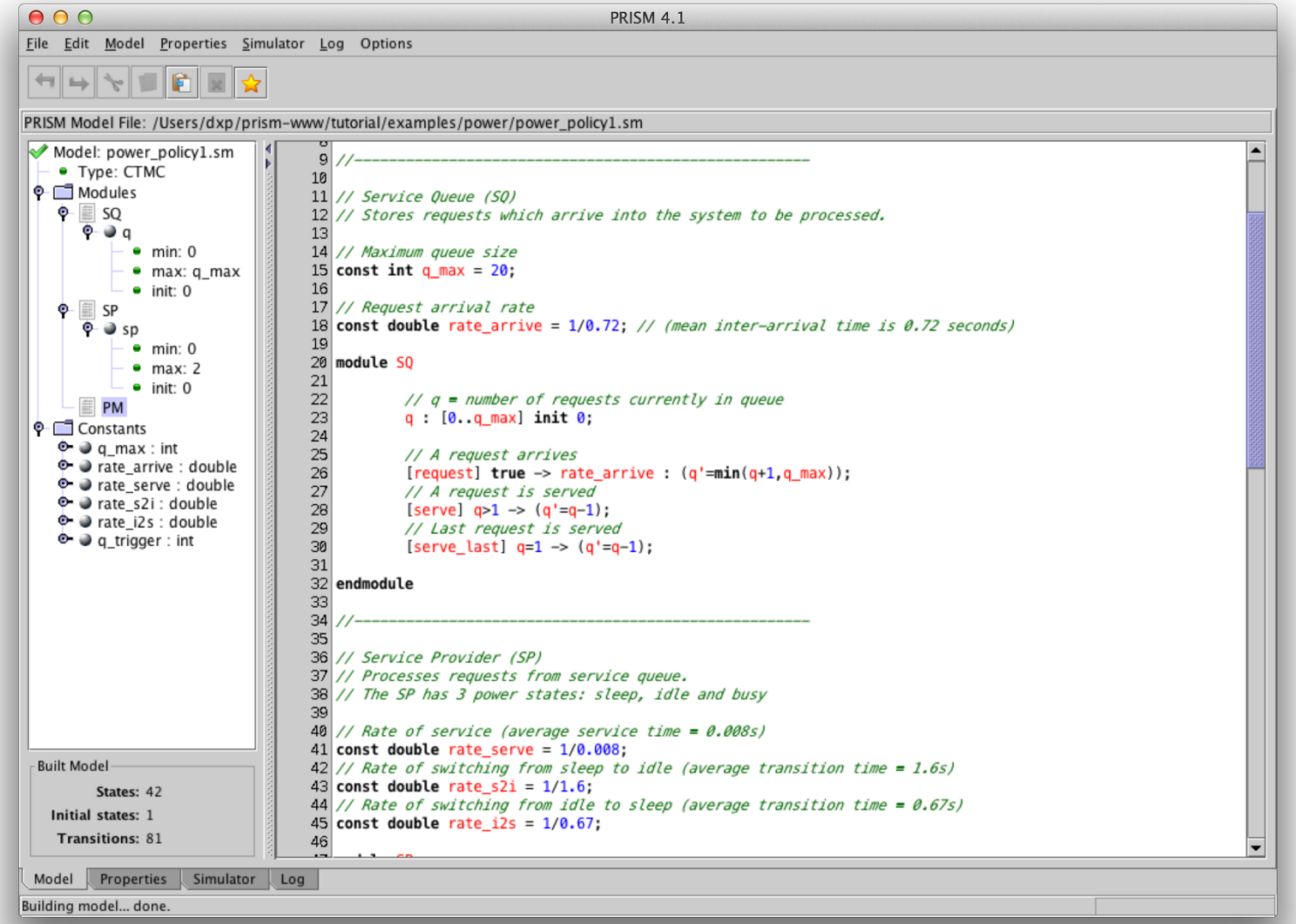
- Usually focus on quantitative (numerical) properties:
  - $P_{=?}$ [F "crash"]
    "what is the probability of a crash occurring?"
  - then analyse trends in quantitative properties as system parameters vary



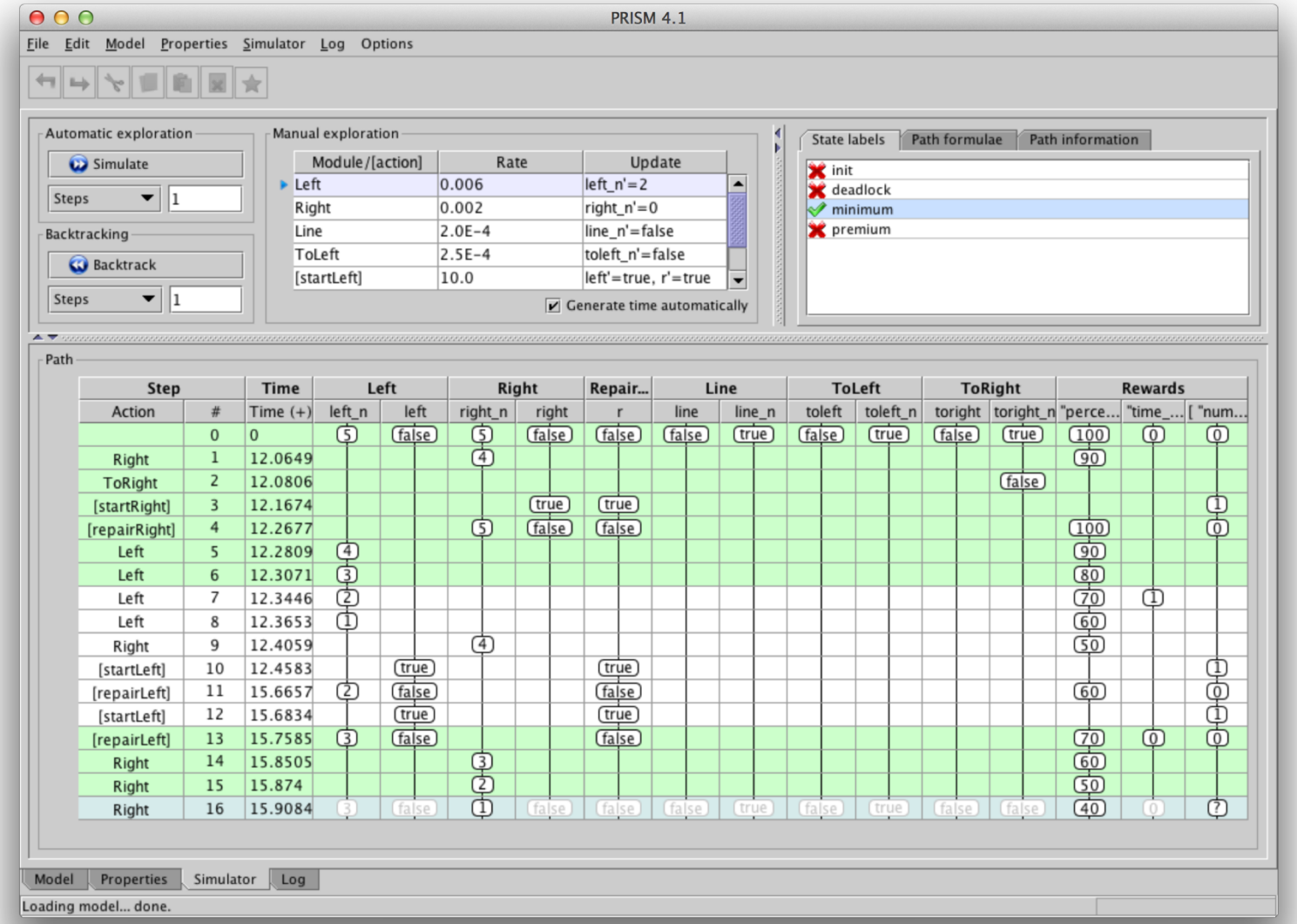
PRISM – Property specification

- Properties can combine numerical + exhaustive aspects
  - $P_{max=?}$ [$F^{\leq 10}$ "fail"] – "worst-case probability of a failure occurring within 10 seconds, for any possible scheduling of system components" (property for MDPs, see Part 2)
  - filter(max, $P_{=?}$ [$G^{\leq 0.02}$!"deploy"], "crash"] – "the maximum probability of an airbag failing to deploy within 0.02s, from any possible crash scenario"
- Reward-based properties (rewards = costs = prices)
  - $R_{\{\text{"time"}\}=?}$ [F "end"] – "expected algorithm execution time"
  - $R_{\{\text{"energy"}\}max=?}$ [$C^{\leq 7200}$] – "worst-case expected energy consumption during the first 2 hours" (for MDPs, see Part 2)
- Properties can be combined with e.g. arithmetic operators
  - e.g. $P_{=?}$ [F $fail_1$] / $P_{=?}$ [F $fail_{any}$] – "conditional failure prob."

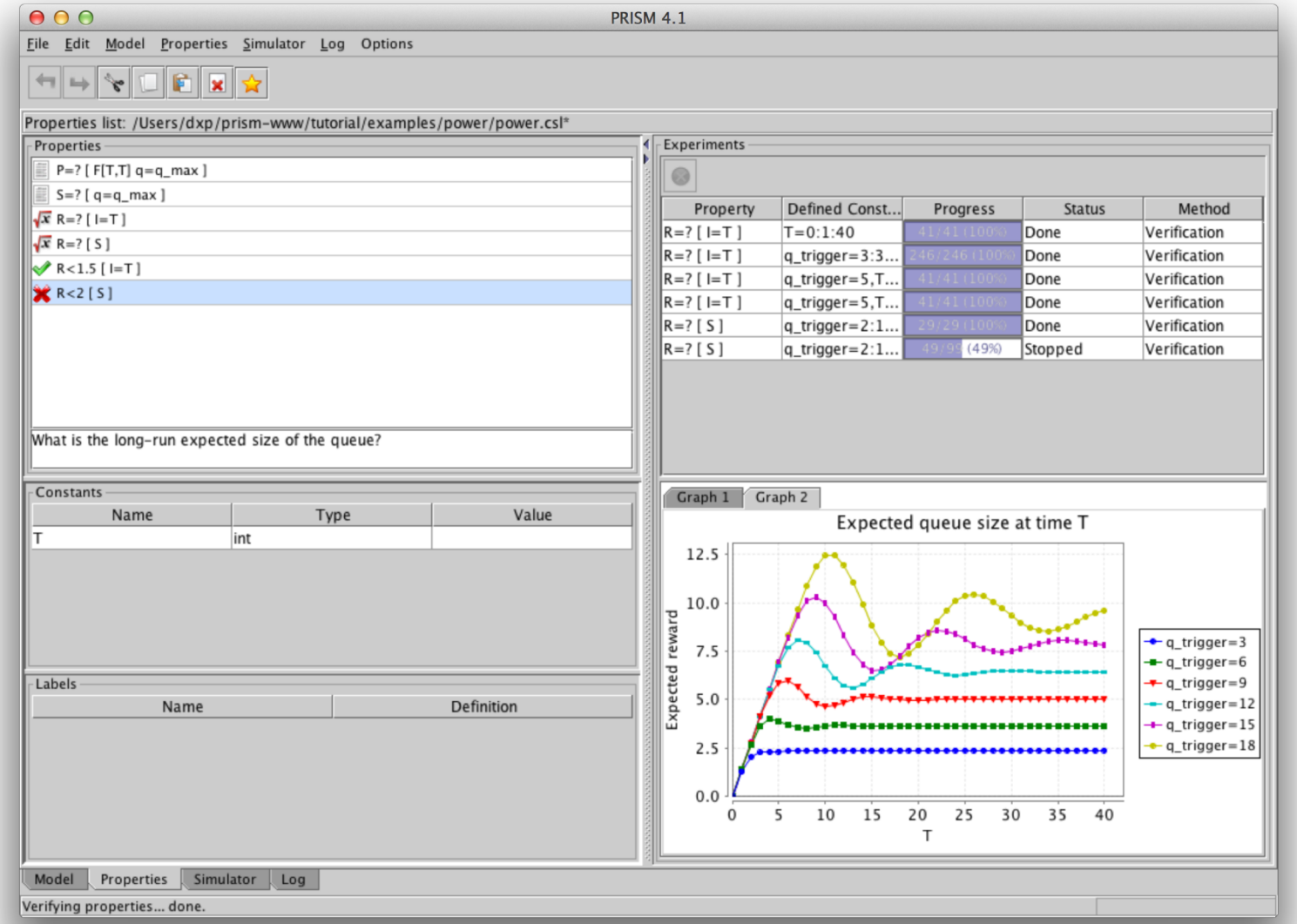
PRISM GUI: Editing a model



PRISM GUI: The Simulator



PRISM GUI: Model checking and graphs



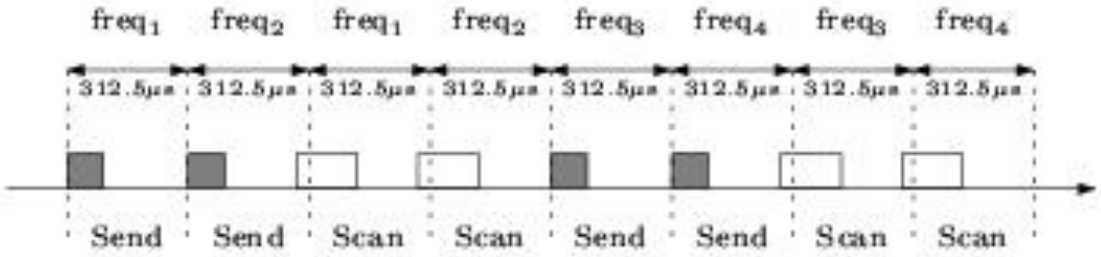
Bluetooth device discovery

- Bluetooth: short-range low-power wireless protocol
  - widely available in phones, PDAs, laptops, ...
  - open standard, specification freely available
- Uses frequency hopping scheme
  - to avoid interference (uses unregulated 2.4GHz band)
  - pseudo-random selection over 32 of 79 frequencies
- Formation of personal area networks (PANs)
  - piconets (1 master, up to 7 slaves)
  - self-configuring: devices discover themselves
- Device discovery
  - mandatory first step before any communication possible
  - relatively high power consumption so performance is crucial
  - master looks for devices, slave listen for master



Master (sender) behaviour

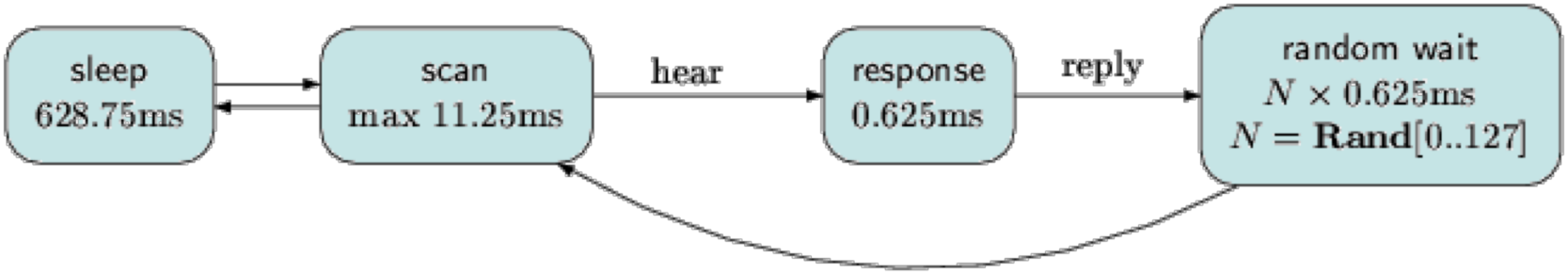
- 28 bit free-running clock CLK, ticks every 312.5µs
- Frequency hopping sequence determined by clock:
  - freq = [CLK_{16-12}+k+ ($CLK_{4-2,0}$– CLK_{16-12}) mod 16] mod 32
  - 2 trains of 16 frequencies (determined by offset k), 128 times each, swap between every 2.56s
- Broadcasts "inquiry packets" on two consecutive frequencies, then listens on the same two



1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
17	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	2	19	20	21	22	23	24	25	26	27	28	29	30	31	32
1	2	3	20	21	22	23	24	25	26	27	28	29	30	31	32
17	18	19	20	5	6	7	8	9	10	11	12	13	14	15	16
17	18	19	20	21	6	7	8	9	10	11	12	13	14	15	16
1	2	3	4	5	6	23	24	25	26	27	28	29	30	31	32
1	2	3	4	5	6	7	24	25	26	27	28	29	30	31	32
17	18	19	20	21	22	23	24	9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24	25	10	11	12	13	14	15	16
1	2	3	4	5	6	7	8	9	10	27	28	29	30	31	32
1	2	3	4	5	6	7	8	9	10	11	28	29	30	31	32
17	18	19	20	21	22	23	24	25	26	27	28	13	14	15	16
17	18	19	20	21	22	23	24	25	26	27	28	29	14	15	16
1	2	3	4	5	6	7	8	9	10	11	12	13	14	31	32
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	32
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
1	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
17	18	3	4	5	6	7	8	9	10	11	12	13	14	15	16
17	18	19	4	5	6	7	8	9	10	11	12	13	14	15	16
1	2	3	4	21	22	23	24	25	26	27	28	29	30	31	32
1	2	3	4	5	22	23	24	25	26	27	28	29	30	31	32
17	18	19	20	21	22	7	8	9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	8	9	10	11	12	13	14	15	16
1	2	3	4	5	6	7	8	25	26	27	28	29	30	31	32
1	2	3	4	5	6	7	8	9	26	27	28	29	30	31	32
17	18	19	20	21	22	23	24	25	26	11	12	13	14	15	16
17	18	19	20	21	22	23	24	25	26	27	12	13	14	15	16
1	2	3	4	5	6	7	8	9	10	11	12	29	30	31	32
1	2	3	4	5	6	7	8	9	10	11	12	13	30	31	32
17	18	19	20	21	22	23	24	25	26	27	28	29	30	15	16
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	16

Slave (receiver) behaviour

- Listens (scans) on frequencies for inquiry packets
  - must listen on right frequency at right time
  - cycles through frequency sequence at much slower speed (every 1.28s)



- On hearing packet, pause, send reply and then wait for a random delay before listening for subsequent packets
  - avoid repeated collisions with other slaves

Bluetooth – PRISM model

- Modelled/analysed using PRISM model checker [DKNP06]
  - model scenario with one sender and one receiver
  - synchronous (clock speed defined by Bluetooth spec)
  - model at lowest-level (one clock-tick = one transition)
  - randomised behaviour so model as a DTMC
  - use real values for delays, etc. from Bluetooth spec

- Modelling challenges
  - complex interaction between sender/receiver
  - combination of short/long time-scales – cannot scale down
  - sender/receiver not initially synchronised, so huge number of possible initial configurations (17,179,869,184)

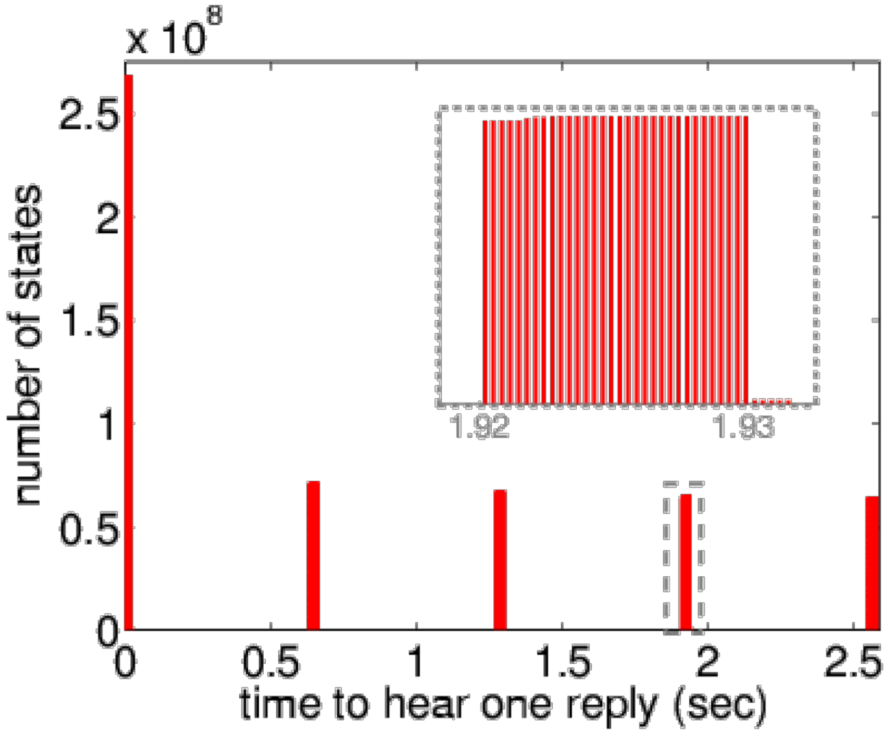
Bluetooth – Results

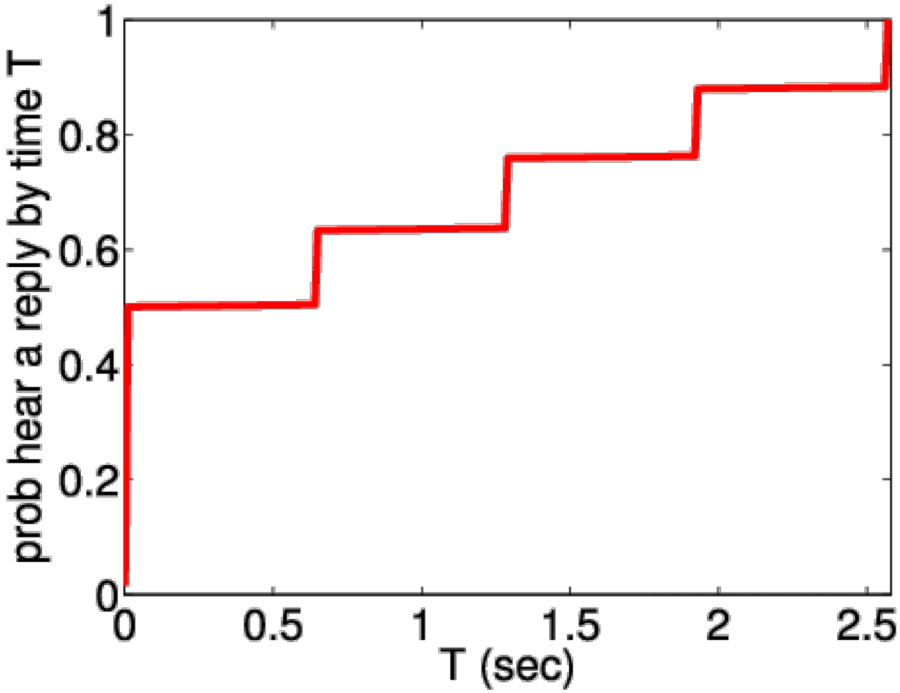
- Huge DTMC – initially, model checking infeasible
  - partition into 32 scenarios, i.e. 32 separate DTMCs
  - on average, approx. 3.4×10^9 states (536,870,912 initial)
  - can be built/analysed with PRISM's MTBDD engine

- We compute:
  - filter(max, R=? [F replies=K], "init"]
  - "worst-case expected time to hear K replies over all possible initial configurations"

- Also look at:
  - how many initial states for each possible expected time
  - cumulative distribution function (CDF) for time, assuming equal probability for each initial state

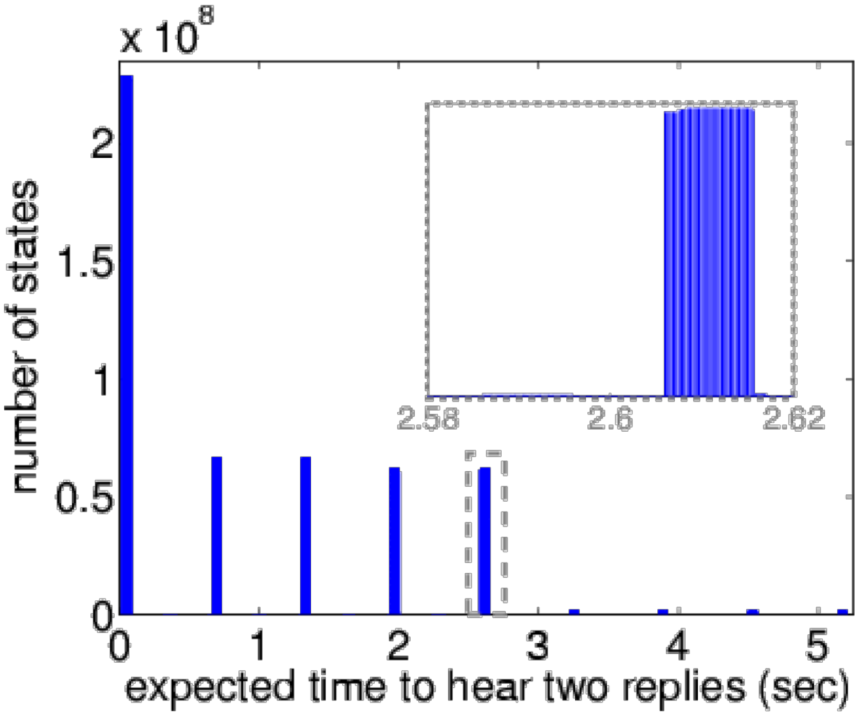
Bluetooth – Time to hear 1 reply

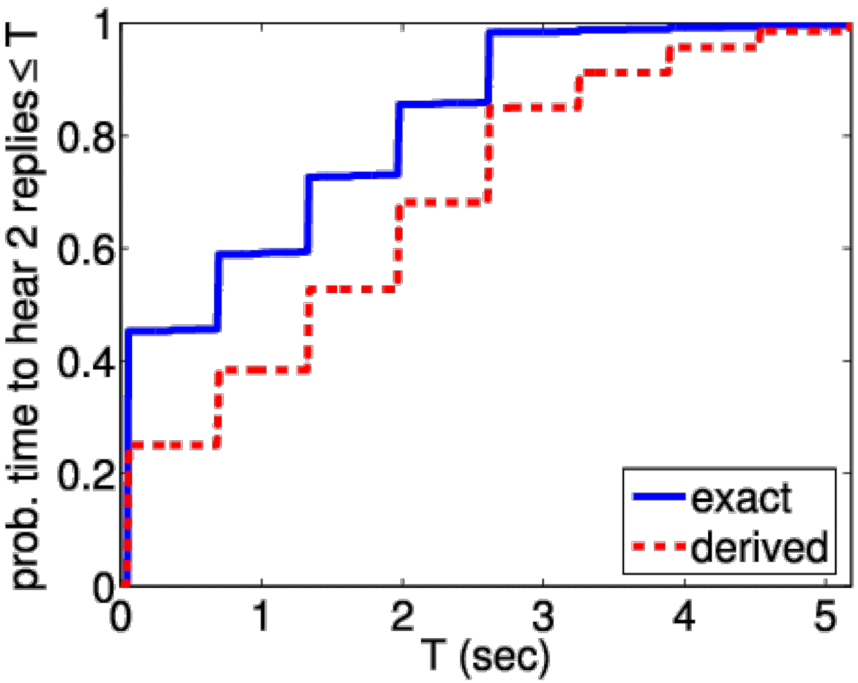




- Worst-case expected time = 2.5716 sec
  - in 921,600 possible initial states
  - best-case = 635 μs

Bluetooth – Time to hear 2 replies

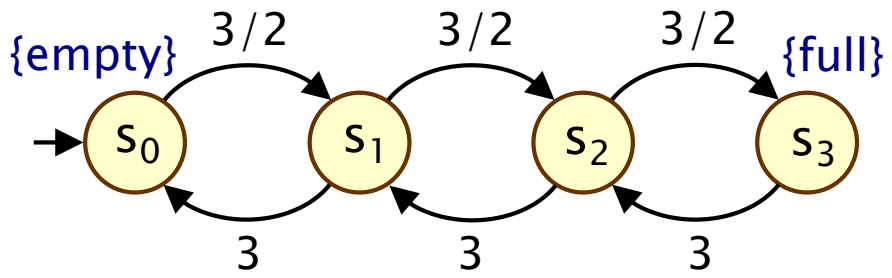




- Worst-case expected time = 5.177 sec
  - in 444 possible initial states
  - compare actual CDF with derived version which assumes times to reply to first/second messages are independent

Beyond DTMCs

- Continuous-time Markov chains
  - transitions taken with real-valued rate (parameter of exponential distribution)
  - suitable for reliability, availability, performance modelling



- Temporal logic CSL – similar to PCTL, except real-valued time
  - $P_{=?}$ [$F^{[4,5.6]}$ outOfPower] – the (transient) probability of being out of power in time interval of 4 to 5.6 time units
  - $S_{=?}$ [minQoS] – the steady-state probability of satisfying minimum QoS
  - $R_{<10}$ [$C^{\leq 5}$] – cumulated reward up to time 5 is less than 10
- Model checking via discretisation (uniformisation)

Summary (Part 1)

- Introduced quantitative verification
  - to analyse path-based properties of probabilistic systems
- Discrete-time Markov chains (DTMCs)
  - state transition systems + discrete probabilistic choice
  - probability space over paths through a DTMC
- Property specifications
  - probabilistic extensions of temporal logic, e.g. PCTL
  - also: expected value of costs/rewards
- Model checking algorithms
  - graph-based algorithms + numerical computation
- Case study: Bluetooth device discovery

- Next: Markov decision processes (MDPs)